



UNIVERSITÉ DE SFAX

FACULTÉ DES SCIENCES ÉCONOMIQUES ET DE GESTION

DÉPARTEMENT D'INFORMATIQUE APPLIQUÉE À LA GESTION

Mémoire de Fin d'Études

**Pour l'obtention de la licence en informatique appliquée à la
gestion**

Plateforme de Financement Collaboratif

Réalisé par : Gdoura Karim

Encadré par : M. Iskandar Keskes

Jury :

Président : _____

Rapporteur : _____

Membre : _____

Année universitaire 2024–2025

Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué à la réalisation de ce travail.

Je remercie tout particulièrement mon encadreur, **M. Iskandar Keskes**.

Mes remerciements s'adressent également aux membres du jury, qui me font l'honneur d'évaluer ce travail. Je les remercie pour leur temps et leur attention.

Je remercie chaleureusement l'équipe pédagogique du département d'informatique appliquée à la gestion pour la qualité de la formation dispensée.

Enfin, je pense à ma famille et à mes amis, qui m'ont soutenu et encouragé tout au long de ce parcours. Leur présence a été une source de motivation constante.

Table des matières

Remerciements	1
Liste des abréviations	9
Introduction générale	10
1 Définition du projet et analyse des solutions existantes	13
1.1 Introduction	13
1.2 Définition de la mission	13
1.2.1 Présentation de l'organisme d'accueil	13
1.2.2 Présentation de l'application	14
1.2.3 Objectifs à atteindre	14
1.3 Contexte du sujet	15
1.4 Analyse d'applications existantes	16
1.4.1 Application A : plateformes internationales de référence	16
1.4.2 Application B : plateformes tunisiennes	17
1.5 Problématique et solution proposée	18
1.6 Conclusion	20
2 Capture des besoins	21
2.1 Introduction	21

2.2	Identification des acteurs du système informatisé	21
2.3	Contexte du système informatisé (boîte noire)	22
2.4	Identification des cas d'utilisation	23
2.5	Description textuelle des cas d'utilisation	25
2.5.1	Inventaire exhaustif des cas d'utilisation	25
2.5.2	Fiches détaillées (format annexe)	27
2.6	Conclusion	32
3	Analyse et conception	33
3.1	Introduction	33
3.2	Réalisation conceptuelle des cas d'utilisation	34
3.2.1	Annuler une contribution	35
3.2.2	Créer et soumettre un projet	37
3.2.3	Investir dans un projet (paiement démo + <i>webhook</i>)	39
3.2.4	Valider un projet (Admin)	41
3.2.5	Fournir les coordonnées bancaires	43
3.2.6	Approuver un retrait (Admin)	45
3.2.7	Mise à jour de l'état d'un projet (Admin)	47
3.3	Modèle du domaine	49
3.4	Architecture du futur logiciel	51
3.4.1	Vue d'ensemble en couches	51
3.4.2	Intégration de n8n	51
3.4.3	Composants asynchrones (BullMQ / Redis)	52
3.4.4	Vue logique en paquetages	52
3.4.5	Vue physique	52
3.4.6	Décisions de conception structurantes	53
3.5	Schémas physiques des données	53

3.5.1	Schéma physique (MongoDB)	53
3.5.2	Justification du modèle NoSQL	54
3.5.3	Indexation et relations	54
3.5.4	Dictionnaire des données	55
3.6	Conclusion	57
4	Implémentation et tests	58
4.1	Introduction	58
4.2	Environnement de réalisation	58
4.2.1	Environnement matériel	58
4.2.2	Environnement logiciel	58
4.2.3	Synthèse des choix technologiques	59
4.3	Réalisation technique des cas d'utilisation	60
4.3.1	Authentification et gestion des utilisateurs	60
4.3.2	Gestion des projets	62
4.3.3	Parcours de paiement (simulation)	64
4.3.4	Nettoyage des investissements abandonnés	65
4.3.5	Moteur de recommandation	66
4.3.6	Service de notifications	67
4.3.7	Stratégie de test	68
4.3.8	Déploiement	69
4.4	Conclusion	69
	Conclusion générale	70
	Webographie	73
	Annexe	74

Format type d'une description textuelle de cas d'utilisation	74
--	----

Table des figures

2.1	Diagramme de contexte du système (boîte noire)	23
2.2	Diagramme de cas d'utilisation	24
3.1	Diagramme de séquence – Annuler une contribution	35
3.2	Diagramme de classes participantes – Annuler une contribution	36
3.3	Diagramme de séquence – Créer et soumettre un projet	37
3.4	Diagramme de classes participantes – Créer et soumettre un projet	38
3.5	Diagramme de séquence – Investir (paiement démo + <i>webhook</i>)	39
3.6	Diagramme de classes participantes – Investir	40
3.7	Diagramme de séquence – Valider un projet	41
3.8	Diagramme de classes participantes – Valider un projet	42
3.9	Diagramme de séquence – Fournir les coordonnées bancaires	43
3.10	Diagramme de classes participantes – Fournir les coordonnées bancaires	44
3.11	Diagramme de séquence – Approuver un retrait (Admin)	45
3.12	Diagramme de classes participantes – Approuver un retrait	46
3.13	Diagramme de séquence – Mise à jour de l'état d'un projet (suspension / réactivation)	47
3.14	Diagramme de classes participantes – Mise à jour de l'état d'un projet	48
3.15	Diagramme de classes du domaine (modèle du domaine)	50
3.16	Diagramme de paquetages de l'application	52
3.17	Architecture générale de l'application	52

4.1	Page de connexion	61
4.2	Formulaire d'inscription avec saisie du code OTP	62
4.3	Formulaire de création de projet (vue porteur)	63
4.4	Console d'administration : validation/rejet d'un projet en UNDER_REVIEW . . .	63
4.5	Formulaire de soutien : saisie du montant et lancement du paiement	65
4.6	Saisie des coordonnées bancaires (chiffrement AES-256-GCM avant stockage) .	65
4.7	Page « Mes investissements » côté investisseur (statuts INITIATED, SUCCESS, FAILED)	66
4.8	Page d'accueil : projets recommandés en fonction du profil de l'investisseur . .	67
4.9	Panneau in-app de notifications (icône cloche, statuts lus / non lus)	68

Liste des tableaux

1.1	Comparaison des plateformes de crowdfunding	19
2.1	Inventaire exhaustif des cas d'utilisation (aligné sur le code livré)	25
2.2	Description textuelle du cas «Créer et soumettre un projet»	28
2.3	Description textuelle du cas «Soutenir un projet»	29
2.4	Description textuelle du cas «Valider / rejeter un projet»	30
2.5	Description textuelle du cas «Voir les projets recommandés»	31
2.6	Description textuelle du cas «Signaler un contenu»	31
3.1	Dictionnaire des données	56
4.1	Technologies utilisées	60
4.2	Modèle de description textuelle d'un cas d'utilisation	74

Liste des abréviations

AES-GCM	Advanced Encryption Standard - Galois/Counter Mode (chiffrement authentifié)
API	Application Programming Interface (interface de programmation)
BCT	Banque Centrale de Tunisie
BullMQ	File de jobs Node.js bâtie sur Redis
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
DLQ	Dead-Letter Queue (file des échecs définitifs)
HMAC	Hash-based Message Authentication Code
HTTPS	Hypertext Transfer Protocol Secure
IA	Intelligence Artificielle
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model (modèle de langage de grande taille)
MVC	Modèle-Vue-Contrôleur
n8n	Outil d'orchestration de workflows
ORM	Object-Relational Mapping (mapping objet-relationnel)
OTP	One-Time Password (code à usage unique)
PFE	Projet de Fin d'Études
RBAC	Role-Based Access Control (contrôle d'accès par rôle)
REST	Representational State Transfer
RGPD	Règlement Général sur la Protection des Données
RIB	Relevé d'Identité Bancaire
SPA	Single Page Application
SSL/TLS	Secure Sockets Layer / Transport Layer Security
TND	Dinar tunisien
UML	Unified Modeling Language

Introduction générale

Contexte général

Le financement participatif (crowdfunding) représente une alternative innovante aux modes de financement traditionnels. En Tunisie, ce secteur connaît un essor important depuis la promulgation de la loi n° 2020-37 du 2 juillet 2020, qui encadre les opérations de financement participatif et établit le cadre réglementaire sous la supervision de la Banque Centrale de Tunisie (BCT).

Cette évolution s'inscrit dans un contexte de transformation numérique de l'économie tunisienne, où les start-ups et PME peinent souvent à accéder aux financements bancaires classiques. Le crowdfunding offre une solution pour mobiliser l'épargne publique au profit de projets entrepreneuriaux innovants, tout en permettant aux investisseurs de diversifier leurs placements.

Problématique

Les plateformes de financement collaboratif existantes présentent plusieurs limitations :

- Absence d'évaluation automatisée des risques pour guider les investisseurs dans leurs choix
- Manque de personnalisation des recommandations de projets selon les profils des investisseurs
- Processus de validation manuelle des projets chronophage pour les administrateurs
- Intégration paiement souvent complexe (webhooks, idempotence, sécurité) et difficile à tester sans sandbox

Face à ces constats, la question de recherche suivante se pose : **Comment concevoir une plateforme de financement collaboratif intégrant l'intelligence artificielle pour automatiser**

l'évaluation des risques et personnaliser les recommandations, tout en garantissant la conformité réglementaire et la sécurité des transactions ?

Objectifs du projet

Ce projet vise à développer une plateforme web de financement participatif répondant aux objectifs suivants :

Objectifs fonctionnels :

1. Permettre aux porteurs de projets de créer, soumettre et gérer leurs campagnes de financement
2. Offrir aux investisseurs une interface intuitive pour découvrir, évaluer et financer des projets
3. Automatiser le processus de validation des projets via une analyse de risque basée sur l'IA
4. Personnaliser les recommandations de projets selon le profil et les préférences de chaque investisseur
5. Proposer un parcours de paiement démonstratif (mock) et une architecture extensible vers un prestataire réel
6. Fournir un tableau de bord administrateur pour superviser les opérations et gérer les litiges

Objectifs techniques :

1. Concevoir une architecture modulaire et scalable (Node.js, React, MongoDB)
2. Implémenter un moteur de recommandation déterministe (profil investisseur, catégories, niveau de risque issu de l'analyse IA des projets, popularité et récence)
3. Intégrer un service LLM pour l'analyse de risque (rapport structuré) via un workflow n8n
4. Orchestrer les workflows asynchrones via n8n (analyse IA) et des files BullMQ (e-mails, reprises)
5. Garantir la résilience du système avec gestion des échecs (dead-letter queues)
6. Assurer la sécurité des données (chiffrement, JWT, HTTPS)

Organisation du mémoire

Ce mémoire est organisé en quatre chapitres. Le premier chapitre fixe le cadre général : mission, contexte du sujet, analyse des solutions existantes, problématique et solution proposée. Le deuxième chapitre formalise la capture des besoins (acteurs, contexte « boîte noire », identification des cas d'utilisation, inventaire exhaustif aligné sur le code, puis fiches détaillées au format annexe pour des cas transverses majeurs). Le troisième chapitre est consacré à l'analyse et à la conception (réalisation conceptuelle des cas d'utilisation, modèle du domaine, architecture du futur logiciel et schémas physiques des données). Le quatrième chapitre décrit l'implémentation et les tests (environnement de réalisation, réalisation technique des cas d'utilisation, stratégie de validation et déploiement). La conclusion générale synthétise les résultats, identifie les apports, les limites et les perspectives d'évolution. Une annexe présente le modèle de fiche utilisé pour les descriptions textuelles de cas d'utilisation (chapitre 2).

Chapitre 1

Définition du projet et analyse des solutions existantes

1.1 Introduction

Ce premier chapitre fixe le cadre général du projet. Il présente la mission confiée (organisme d'accueil, application à réaliser, objectifs visés), situe le sujet dans son contexte économique et réglementaire, analyse les principales solutions existantes sur le marché du financement participatif, puis dégage la problématique et la solution proposée. Il fournit ainsi les éléments nécessaires à la compréhension des besoins formalisés dans le chapitre suivant.

1.2 Définition de la mission

1.2.1 Présentation de l'organisme d'accueil

Ce projet de fin d'études s'inscrit dans le cadre d'un stage académique encadré par M. Iskandar Keskes, enseignant à la Faculté des Sciences Économiques et de Gestion (FSEG) de Sfax, département d'Informatique Appliquée à la Gestion. Le travail a été réalisé en autonomie, avec des points de suivi hebdomadaires permettant de valider les choix de conception, de cadrer les priorités et d'arbitrer les compromis fonctionnels et techniques. Le contexte académique a privilégié une approche orientée *ingénierie des systèmes d'information* : capture des besoins, modélisation UML, conception des données, réalisation et validation par scénarios métier,

conformément aux attendus pédagogiques de la licence en Informatique Appliquée à la Gestion.

1.2.2 Présentation de l'application

L'application réalisée est une **plateforme web de financement collaboratif** (*crowdfunding*) qui met en relation trois catégories d'acteurs métier :

- les **porteurs de projet** qui souhaitent financer une campagne (titre, description, catégorie, objectif financier, échéance, médias) ;
- les **investisseurs** qui souhaitent contribuer à des campagnes correspondant à leur profil (catégories préférées, tolérance au risque) ;
- les **administrateurs** qui assurent la gouvernance de la plateforme (validation des projets, modération des contenus, traitement des signalements, supervision des retraits).

Les concepts métier centraux portés par l'application sont les suivants :

- **Projet** : campagne de financement caractérisée par un cycle de vie explicite (DRAFT, AWAITING_AI, UNDER_REVIEW, APPROVED, ACTIVE, REJECTED, FUNDED, CLOSED, SUSPENDED).
- **Investissement** : engagement d'un investisseur pour un montant donné sur un projet, créé en INITIATED puis confirmé en SUCCESS (ou REFUNDED en cas de sur-financement) après réception du *webhook* de paiement.
- **Analyse de risque** : rapport structuré (résumé, avantages, inconvénients, améliorations, éléments à retirer, questions, score/riskLevel) produit par un service LLM orchestré par n8n, et destiné à éclairer la revue administrative.
- **Recommandations personnalisées** : classement des projets accessibles à un investisseur en fonction de ses catégories préférées et de sa tolérance au risque.
- **Notifications** : notifications *in-app* systématiques (lecture par l'interface) et e-mails déclenchés sur les seuls événements critiques (validation/rejet, contribution, paiement, retrait, désactivation), afin de limiter le bruit.
- **Retrait (*payout*)** : flux de versement au porteur de projet une fois la campagne FUNDED ou CLOSED, après saisie chiffrée des coordonnées bancaires et approbation administrative.

1.2.3 Objectifs à atteindre

La plateforme doit répondre aux objectifs suivants, formulés à la fois sur le plan métier et sur le plan technique.

Objectifs métier :

- Permettre aux porteurs de projets de créer, soumettre, publier et suivre leurs campagnes de financement.
- Offrir aux investisseurs une expérience claire de découverte, d'évaluation et de contribution aux projets.
- Automatiser l'évaluation préliminaire du risque pour fiabiliser et accélérer la revue administrative.
- Personnaliser les recommandations selon le profil de l'investisseur (catégories, tolérance au risque).
- Assurer la traçabilité des transactions (statuts d'investissement, de transaction et de *payout*).
- Outiller la gouvernance de la plateforme (validation, modération, signalements, retraits, statistiques).

Objectifs techniques :

- Concevoir une architecture modulaire (Node.js / Express + React + MongoDB) avec une séparation nette entre logique métier et accès aux données.
- Mettre en place un parcours de paiement de démonstration (*mock*) avec confirmation par *webhook* signé et idempotent, extensible vers un prestataire réel.
- Orchestrer les traitements asynchrones (analyse IA, notifications, reprises) via n8n et des files BullMQ/Redis, avec stockage des échecs définitifs (*dead-letter queues*).
- Garantir la sécurité (authentification par JWT en cookies HttpOnly, contrôle d'accès par rôle, *rate limiting*, signature des *webhooks*, chiffrement AES-256-GCM des coordonnées bancaires).

1.3 Contexte du sujet

Le financement participatif (*crowdfunding*) constitue une alternative innovante aux modes de financement bancaires traditionnels. En Tunisie, le secteur connaît un essor important depuis la promulgation de la **loi n° 2020-37 du 2 juillet 2020**, qui encadre les opérations de financement participatif et place leur supervision sous la tutelle de la Banque Centrale de Tunisie (BCT). Ce cadre réglementaire ouvre la voie à des plateformes qui mobilisent l'épargne publique au profit de projets entrepreneuriaux, tout en imposant des exigences fortes en matière d'identification des acteurs, de traçabilité des opérations et de conservation des journaux financiers.

Sur le plan économique, ce mouvement répond à une difficulté concrète des start-ups et PME tunisiennes pour accéder aux financements bancaires classiques, en particulier pour les projets

innovants ou portant un risque élevé. Sur le plan technologique, l'émergence des modèles de langage (LLM) et des outils d'orchestration de *workflows* (par exemple n8n) permet d'automatiser une partie de la pré-évaluation des projets et de fiabiliser les flux asynchrones (paiements, notifications, reprises) sans alourdir la base de code applicative. Le sujet du présent projet s'inscrit donc à l'intersection de la **conformité réglementaire**, de l'**ingénierie des systèmes d'information** et de l'**IA appliquée** à la décision métier.

1.4 Analyse d'applications existantes

Cette section présente une étude comparative des principales plateformes de financement participatif, en distinguant les acteurs internationaux qui ont structuré le marché et les acteurs locaux qui en illustrent la déclinaison tunisienne. L'objectif est d'identifier les fonctionnalités déjà offertes et, surtout, les manques que la solution proposée vise à combler.

1.4.1 Application A : plateformes internationales de référence

Kickstarter

Kickstarter est la plateforme pionnière du crowdfunding, lancée en 2009. Elle se concentre sur le financement de projets créatifs (arts, design, technologie). Le modèle "tout ou rien" (all-or-nothing) impose que le projet atteigne 100% de son objectif pour que les fonds soient débloqués.

Points forts :

- Grande notoriété et trafic important
- Communauté engagée de créateurs
- Processus de validation qualité rigoureux

Points faibles :

- Absence d'analyse de risque automatisée
- Pas de personnalisation des recommandations
- Frais élevés (5% + 3-5% frais de paiement)

Indiegogo

Indiegogo offre plus de flexibilité que Kickstarter avec deux modèles : “tout ou rien” et “flexible funding” (les fonds sont conservés même si l’objectif n’est pas atteint).

Points forts :

- Flexibilité du modèle de financement
- Support de campagnes en continu (pas de deadline obligatoire)
- Partenariats avec des fabricants (InDemand)

Points faibles :

- Taux de succès des campagnes souvent plus faible que les leaders du marché (données publiées hétérogènes)
- Interface moins intuitive
- Manque de filtrage intelligent

1.4.2 Application B : plateformes tunisiennes

CoFundy

CoFundy est une plateforme tunisienne de crowdfunding lancée en 2016, spécialisée dans le financement de projets entrepreneuriaux.

Caractéristiques :

- Intégration avec paiements tunisiens (cartes bancaires locales)
- Validation manuelle des projets (délai 7-10 jours)
- Frais : 5% du montant collecté

Limitations identifiées :

- Absence de recommandations personnalisées
- Interface utilisateur datée
- Pas d’évaluation de risque visible pour les investisseurs
- Intégration paiement souvent limitée / difficile à tester sans sandbox

Kaoun

Plateforme tunisienne plus récente (2019), focalisée sur les projets sociaux et environnementaux.

Points forts :

- Niche bien définie (impact social)
- Communauté engagée

Points faibles :

- Volume de projets limité
- Fonctionnalités basiques
- Pas d'IA ou d'automatisation

1.5 Problématique et solution proposée

La revue des plateformes existantes met en évidence quatre manques structurels qui pénalisent à la fois les investisseurs, les porteurs de projet et les administrateurs.

Manques observés :

1. Absence d'évaluation automatisée des risques pour guider les investisseurs et accélérer la revue.
2. Manque de personnalisation des recommandations selon le profil de l'investisseur (catégories, tolérance au risque).
3. Processus de validation des projets manuels et chronophages pour les administrateurs.
4. Intégration paiement souvent peu transparente, difficile à tester sans environnement de bac à sable, et faiblement résiliente face aux échecs (annulations, remboursements, sur-financement).

Le tableau 1.1 synthétise ces écarts et positionne la solution proposée par rapport aux plateformes étudiées.

Critère	Kickstarter	Indiegogo	CoFundy	Kaoun	Notre solution
Analyse IA	Non	Non	Non	Non	Oui
Recommandations	Non	Basique	Non	Non	Personnalisées
Païement TN	Non	Non	Limité	Oui	Démo (mock) / extensible
Gestion des échecs	Manuelle	Manuelle	Manuelle	Manuelle	Automatisée (retries + DLQ)
Frais	8–10%	8–13%	5%	5%	— (hors périmètre du prototype PFE)

TABLE 1.1 – Comparaison des plateformes de crowdfunding

Question de recherche. Comment concevoir une plateforme de financement collaboratif intégrant l’intelligence artificielle pour automatiser l’évaluation des risques et personnaliser les recommandations, tout en garantissant la conformité réglementaire (BCT) et la sécurité des transactions ?

Solution proposée. La plateforme développée combine quatre choix structurants :

- Une **analyse de risque automatisée** (LLM orchestré par n8n) qui produit un rapport structuré (résumé, avantages, inconvénients, améliorations, éléments à retirer, questions, score) lors de la soumission d’un projet, avec **auto-rejet** lorsque une heuristique de **cohérence budgétaire** (objectif fundingGoal vs estimation des besoins extraits de la description) détecte un **écart absolu $\geq 30\%$** (règle bloquante), et relances automatiques en cas d’indisponibilité du workflow.
- Un **moteur de recommandations** entièrement déterministe dans le code livré (catégories préférées, adéquation au riskLevel issu de l’analyse IA du projet, progression et récence de la campagne), extensible ultérieurement par enrichissement LLM.
- Un **parcours de paiement de démonstration** (mock) avec confirmation par *webhook* signé et idempotent, conçu pour être remplacé par un prestataire réel sans changer la logique métier.
- Une **gestion résiliente des échecs** via files BullMQ/Redis et workflows n8n, avec stockage des événements définitivement échoués (*dead-letter queues*) pour reprise manuelle par l’administrateur.

1.6 Conclusion

Ce chapitre a permis de définir la mission (organisme d'accueil, application à réaliser, objectifs métier et techniques), de situer le sujet dans son contexte économique et réglementaire, d'analyser les principales plateformes existantes et d'en dégager la problématique. La solution proposée s'appuie sur l'analyse IA, des recommandations personnalisées, un paiement de démonstration extensible et une orchestration résiliente. Le chapitre suivant capture formellement les besoins métier sous forme d'acteurs et de cas d'utilisation.

Chapitre 2

Capture des besoins

2.1 Introduction

Ce chapitre formalise les besoins métier de la plateforme du point de vue des acteurs qui l'utilisent. Il décrit le périmètre du système (boîte noire), identifie les acteurs et leurs rôles, recense les cas d'utilisation regroupés en paquetages métier, dresse un **inventaire exhaustif** de ces cas (tableau 2.1), puis présente des **descriptions détaillées au format annexe** pour un sous-ensemble **structurant** (nominal, alternatifs, exceptions) en corrélation avec les modules `/api/*` et `/internal/*` du backend livré.

2.2 Identification des acteurs du système informatisé

La plateforme distingue des acteurs humains et des acteurs systèmes externes. Un même utilisateur peut cumuler les rôles d'*investisseur* et de *porteur de projet* ; la distinction de ces deux rôles est portée dans la description des cas d'utilisation.

- **Visiteur** : utilisateur non authentifié ; consulte et recherche les projets publics, peut créer un compte et lancer une procédure de récupération de mot de passe.
- **Utilisateur authentifié** (porteur de projet et / ou investisseur) : gère son compte, crée et soumet des projets, contribue à des campagnes, commente et signale des contenus, suit ses investissements, ses paiements et ses retraits.
- **Administrateur** : valide, publie, suspend ou rejette les projets, modère les commentaires, traite les signalements, supervise les retraits, désactive ou réactive les comptes, et relance les traitements échoués (notifications, retraits, remboursements).

- **Passerelle de paiement (d  mo)** : partenaire externe simul   qui confirme ou infirme les paiements via un *webhook* sign  . Les messages de ce tiers d  clenchent des transitions m  tier (statut de transaction, statut d'investissement, mise    jour du montant collect  , remboursements en cas de sur-financement).
- **Service d'analyse de risque (orchestrateur)** : service externe qui ex  cute les workflows d'analyse (sollicitation, reprises, mise    jour du rapport) et retourne un rapport structur   via les routes internes du backend (`/internal/*`); l'assistance conversationnelle sur un projet est, quant    elle, expos  e par l'API `/api/projects/:id/chat` (orchestration de type n8n / mod  le de langage pour l'analyse).
- **Service de messagerie (SMTP)** : partenaire externe utilis   pour les e-mails critiques (v  rification, r  initialisation, d  cisions admin,   v  nements de paiement / retrait).

2.3 Contexte du syst  me informatis   (bo  te noire)

Vue de l'ext  rieur, la plateforme   change des messages avec **quatre familles de partenaires** (canal web humain, passerelle de paiement, service d'analyse de risque, messagerie). La figure 2.1 mod  lise ce contexte : le syst  me est repr  sent   comme une « bo  te noire » au centre et chaque acteur y est reli   par des associations libell  es (  changes fonctionnels).

- Les **acteurs humains web** (visiteur, utilisateur authentifi  , administrateur) interagissent avec le syst  me via l'application front-end (HTTPS, REST, cookies HttpOnly).
- La **passerelle de paiement (d  mo)** envoie des confirmations sign  es (*webhook*); le syst  me r  pond de fa  on idempotente pour absorber les retentatives.
- Le **service d'analyse de risque (orchestrateur)** consomme des demandes d'analyse (via file de jobs/Redis) et appelle des routes internes du backend (`/internal/*`) pour mettre    jour les r  sultats (succ  s/  chec) de mani  re contr  l  e.
- Le **service de messagerie (SMTP)** est sollicit   pour les e-mails critiques (v  rification, r  initialisation, d  cisions admin, paiement/retrait).

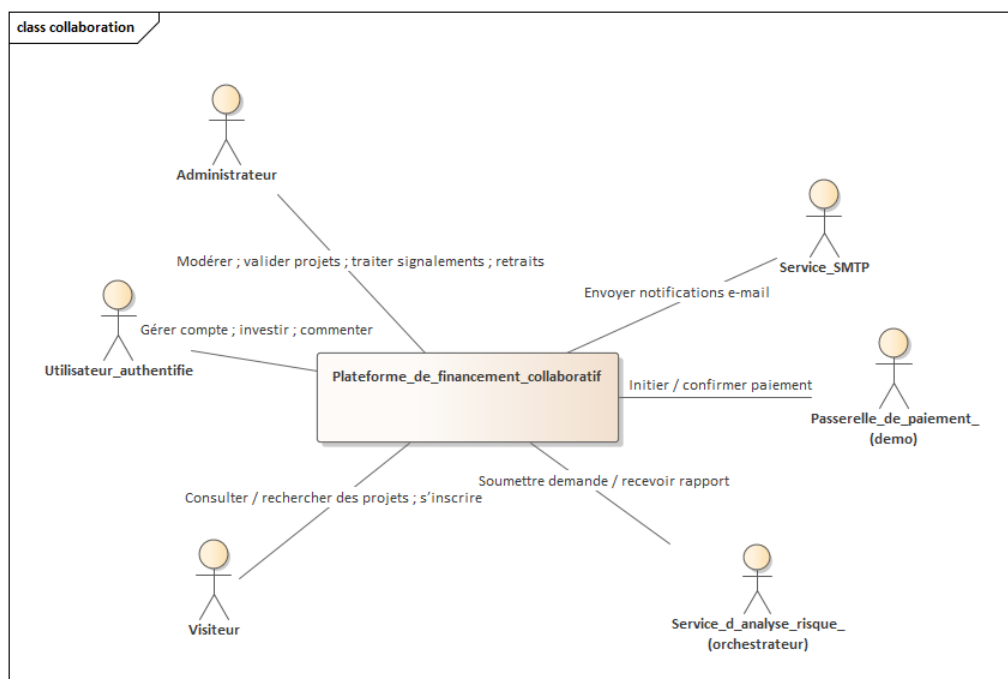


FIGURE 2.1 – Diagramme de contexte du système (boîte noire)

Le périmètre du logiciel est ainsi délimité : tout ce qui n'appartient pas au cœur applicatif (persistance métier, API et interface utilisateur de la plateforme) relève d'un partenaire externe et fait l'objet d'un contrat d'interface (REST, *webhook* signé, appels internes documentés).

2.4 Identification des cas d'utilisation

Les cas d'utilisation sont regroupés en thèmes métier qui structurent la plateforme (présentation synthétique). Les détails d'enchaînement et les variantes sont explicités dans les descriptions textuelles (section 2.5) et dans les diagrammes du chapitre 3.

- **Comptes** : inscription (avec OTP / lien de vérification), connexion, déconnexion, rafraîchissement de session, mot de passe oublié, mise à jour du profil, suppression de compte.
- **Projets** : exploration / consultation, recherche, création (DRAFT), soumission à l'analyse IA (AWAITING_AI), revue administrative (UNDER_REVIEW), décision (APPROVED/REJECTED), publication (ACTIVE), édition / re-soumission, archivage.
- **Contributions & finance** : contribution (paiement simulé), suivi des investissements, relance de paiement, annulation dans la fenêtre de grâce, remboursement en cas de

sur-financement, retraits (*payouts*) et saisie chiffrée des coordonnées bancaires.

- **Échanges & confiance** : commentaires sur les projets publics (publication/suppression), notifications *in-app*, signalement (projet ou commentaire), consultation de ses signalements, recommandations personnalisées, chatbot d'aide à la consultation d'un projet.
- **Administration** : gestion des projets (liste, validation/rejet, publication, suspension/réactivation, relance analyse IA), gestion des utilisateurs (désactivation/réactivation), modération des commentaires (masquer/rétablir), traitement des signalements (résolution), supervision des retraits (approbation), relance des événements en échec (notifications, retraits, remboursements).

La représentation graphique synthétique de ces cas d'utilisation est donnée par la figure 2.2.

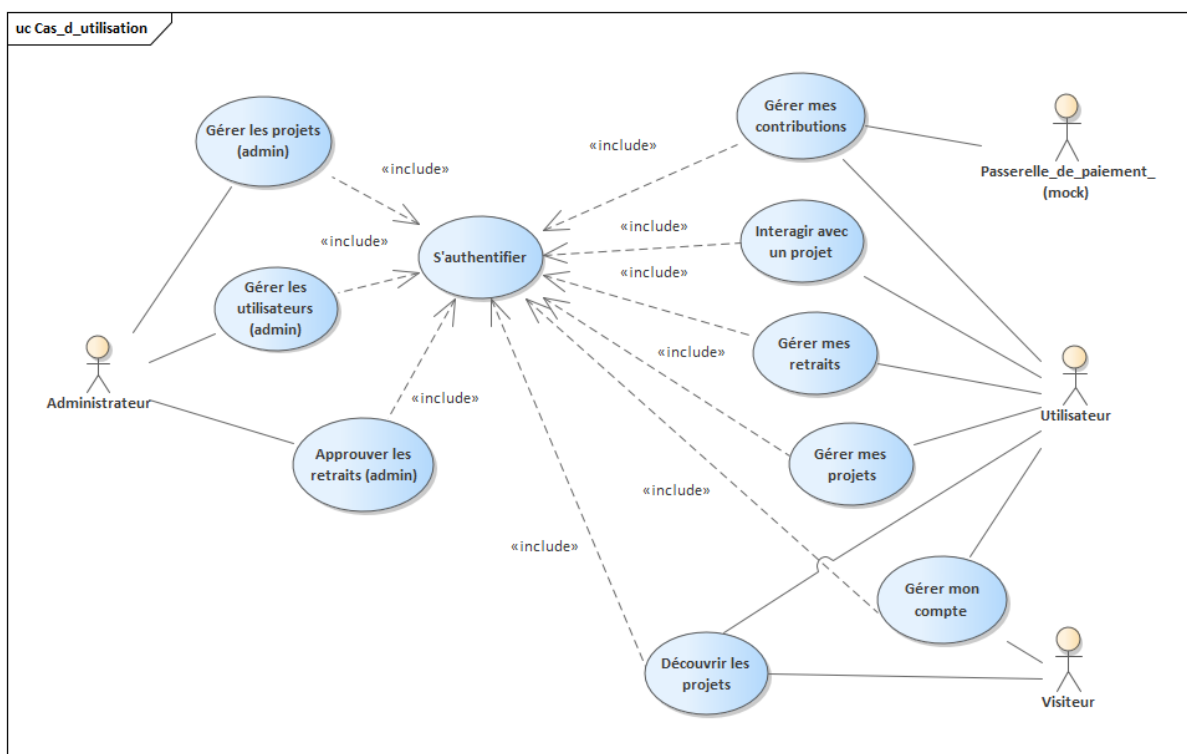


FIGURE 2.2 – Diagramme de cas d'utilisation

Lecture du diagramme. Le diagramme positionne les acteurs autour du système et regroupe les cas d'utilisation par thèmes métier. Les liens «include» (lorsqu'ils sont utilisés) permettent de factoriser des sous-fonctionnalités obligatoires et réutilisées (par exemple l'authentification), tandis que les variantes sont détaillées dans les scénarios textuels (section 2.5) et dans les diagrammes d'interaction du chapitre 3.

Alignement avec l'implémentation. La couverture fonctionnelle du diagramme correspond aux modules exposés par le backend (/api/auth, /api/projects, /api/investments, /api/webhooks, /api/payouts, /api/notifications, /api/reports, /api/recommendations, /api/projects/:id/chat (chatbot), /api/admin, /internal/*). Le tableau 2.1 en donne la cartographie exhaustive ; le chapitre 3 en reprend une partie sous forme UML.

2.5 Description textuelle des cas d'utilisation

Conformément au plan du rapport (version 2), la capture des besoins exige à la fois (i) un inventaire exhaustif des cas d'utilisation aligné sur le diagramme de la section 2.4 et sur le code livré, et (ii) des fiches détaillées (acteur, préconditions, postconditions, nominal, variantes, exceptions) pour les cas transverses les plus structurants. Le modèle de fiche est rappelé en **Annexe** (tableau 4.2).

2.5.1 Inventaire exhaustif des cas d'utilisation

Le tableau 2.1 recense, par paquetage métier, les capacités exposées par l'application (front-end React, API REST /api/*, webhooks /api/webhooks, orchestration /internal/* protégée, files BullMQ/Redis). Il constitue la trace « exhaustive » attendue par le plan (version 2) au sens **fonctionnel** ; les fiches détaillées du §2.5.2 en approfondissent cinq représentants.

TABLE 2.1 – Inventaire exhaustif des cas d'utilisation (aligné sur le code livré)

Paquetage	Cas d'utilisation	Acteurs / surfaces logicielles
Comptes	Inscription et vérification du compte (OTP et/ou lien e-mail)	Visiteur → Utilisateur ; /api/auth ; SMTP
Comptes	Connexion, déconnexion, rotation des jetons (<i>refresh</i>)	Utilisateur, Administrateur ; /api/auth ; cookies HttpOnly
Comptes	Mot de passe oublié / réinitialisation sécurisée	Visiteur/Utilisateur ; /api/auth ; SMTP
Comptes	Consultation et mise à jour du profil (préférences de risque, catégories)	Utilisateur ; /api/users
Comptes	Suppression de compte (contraintes si investissement actif)	Utilisateur ; /api/users

Paquetage	Cas d'utilisation	Acteurs / surfaces logicielles
Projets	Exploration, recherche et consultation d'un projet public	Visiteur, Utilisateur; /api/projects/public, /api/projects/search, optionalAuth
Projets	Création, modification, archivage d'un brouillon (DRAFT)	Porteur de projet; /api/projects
Projets	Soumission à l'analyse IA (AWAITING_AI) et suivi des relances	Porteur; n8n/BullMQ; /internal/run-risk-analysis (routes internes authentifiées)
Projets	Décision administrative (UNDER_REVIEW → APPROVED/REJECTED), publication (ACTIVE)	Administrateur; /api/admin/projects
Projets	Suspension / réactivation, relance d'analyse IA	Administrateur; /api/admin/projects
Projets	Révoquer une approbation (APPROVED → REJECTED) pour demander des corrections	Administrateur; /api/admin/projects/.../revoke-approval
Finance	Initier une contribution, paiement simulé, webhook signé et idempotent	Investisseur, passerelle; /api/investments, /api/webhooks
Finance	Annulation pendant la fenêtre de grâce (CANCELLING/CANCELLED)	Investisseur; /api/investments/.../cancel
Finance	Liste « mes investissements », relance de paiement, suivi des transactions	Investisseur; collections investments, transactions
Finance	Remboursements (REFUNDED), anti-sur-financement, relances admin d'échecs	Système, administrateur; services métier + files
Finance	Saisie chiffrée des coordonnées bancaires (AES-256-GCM)	Porteur de projet; PUT /api/payouts/:id/bank-details
Finance	Demande de retrait (payout) et validation administrative	Porteur, administrateur; /api/payouts, /api/admin
Confiance	Commentaires sur projet public (publication, suppression auteur)	Utilisateur; /api/projects/.../comments

Paquetage	Cas d'utilisation	Acteurs / surfaces logicielles
Confiance	Signalement projet ou commentaire ; consultation « mes signalements »	Utilisateur ; /api/reports
Confiance	Traitement des signalements (RESOLVED/DISMISSED)	Administrateur ; /api/admin/reports
Confiance	Notifications in-app et e-mails asynchrones (événements critiques)	Tous rôles ; /api/notifications ; BullMQ
Confiance	Recommandations (score déterministe : catégories, riskLevel, progression, récence)	Utilisateur ; /api/recommendations
Confiance	Chatbot d'aide contextualisé sur un projet (LLM)	Utilisateur ; POST /api/projects/:id/chat
Administration	Désactivation / réactivation d'utilisateurs	Administrateur ; /api/admin/users
Administration	Modération des commentaires (masquage / rétablissement)	Administrateur ; /api/admin/comments
Administration	Supervision des files d'échec (notifications, payouts, remboursements, annulations)	Administrateur ; collections failed* + actions de relance

2.5.2 Fiches détaillées (format annexe)

Les tableaux 2.2 à 2.6 détaillent cinq cas transverses (création de projet, contribution, validation administrative, recommandations, signalement). Les autres lignes du tableau 2.1 suivent le même canevas en annexe ou en fiches séparées si le jury l'exige.

Cas d'utilisation	Créer et soumettre un projet
Acteur	Utilisateur (porteur de projet)
Précondition	L'utilisateur est authentifié
Postcondition	Projet persisté ; statut DRAFT puis AWAITING_AI après soumission ; analyse IA planifiée
Scénario nominal	1. L'utilisateur renseigne titre, description, catégorie, objectif (<code>fundingGoal</code> $\geq$ 1), fenêtre temporelle (<code>startAt</code>, <code>deadline</code>), médias éventuels. 2. Le système enregistre un projet en DRAFT. 3. L'utilisateur déclenche la soumission : passage en AWAITING_AI, création d'un job (trace <code>aiJobId</code>, compteurs de relance). 4. Le workflow n8n/LLM produit ou enrichit <code>project.aiAnalysis</code> (rapport + <code>score/riskLevel</code>). 5. Si aucune règle bloquante n'est déclenchée, le projet passe en UNDER_REVIEW et l'administration est notifiée.
Scénarios alternatifs	1. Auto-rejet budgétaire : heuristique <code>riskHeuristicService</code> — si l'écart entre l'objectif et l'estimation des besoins extraits de la description atteint ou dépasse 30 % (sévérité BLOCK), passage immédiat en REJECTED avec motif structuré (<code>/internal/workflow</code>). 2. Échec ou indisponibilité du pipeline IA : le projet reste en AWAITING_AI ; relances automatiques (<code>aiAutoRetryCount</code>, <code>aiAnalysisRetries</code> bornées).
Exceptions	1. Validation serveur (champs obligatoires, dates, médias). 2. <code>fundingGoal</code> invalide (< 1) : rejet 400.

TABLE 2.2 – Description textuelle du cas « Créer et soumettre un projet »

Cas d'utilisation	Soutenir un projet (paiement simulé)
Acteur	Utilisateur (investisseur), Passerelle de paiement
Précondition	Utilisateur authentifié ; projet ACTIVE ; montant $\geq$ minimum applicatif (100 TND côté API)
Postcondition	Investissement INITIATED puis SUCCESS si <i>webhook</i> SUCCEDED ; incrément atomique de <i>currentFunding</i>
Scénario nominal	1. L'utilisateur saisit un montant $\geq$ 100 TND. 2. Le système crée un investissement (INITIATED) et une transaction (PENDING, couple <i>provider+providerPaymentId</i> unique). 3. Le système renvoie une URL de paiement simulée. 4. La passerelle appelle le <i>webhook</i> signé. 5. Le backend vérifie signature + idempotence. 6. Transaction SUCCEDED, investissement SUCCESS, mise à jour du total collecté (garde-fou anti-sur-financement).
Scénarios alternatifs	1. Paiement refusé : transaction FAILED / CANCELLED, investissement FAILED. 2. Sur-financement : statuts REFUNDED, remontées admin pour relance éventuelle. 3. Annulation utilisateur dans la fenêtre de grâce (CANCELLING $\rightarrow$ CANCELLED).
Exceptions	1. Projet indisponible ou investisseur administrateur (règle <i>requireNotAdmin</i>). 2. Montant hors bornes (<100 TND).

TABLE 2.3 – Description textuelle du cas « Soutenir un projet »

Cas d'utilisation	Valider / rejeter un projet
Acteur	Administrateur
Précondition	Authentification admin ; projet en UNDER_REVIEW (sauf scénario de révocation)
Postcondition	Projet APPROVED ou REJECTED ; journalisation ; notifications créateur
Scénario nominal	1. L'administrateur consulte la file des projets en revue. 2. Il analyse le rapport IA (aiAnalysis.report, scores). 3. Il émet une décision documentée (APPROVED/REJECTED). 4. Le service adminProjectService applique la transition autorisée (projectLifecycle). 5. Notifications in-app + e-mail au créateur.
Scénarios alternatifs	1. Projet REJECTED : le créateur peut corriger et resoumettre (DRAFT → cycle). 2. Révocation d'approbation : depuis APPROVED, l'administrateur appelle /api/admin/projects/:id/revoke-approval ; retour REJECTED avec demande de corrections.
Exceptions	1. Identifiant inconnu ou droits insuffisants. 2. Transition interdite (ex. publication sans approbation).

TABLE 2.4 – Description textuelle du cas « Valider / rejeter un projet »

Cas d'utilisation	Voir les projets recommandés
Acteur	Utilisateur (investisseur)
Précondition	Utilisateur authentifié
Postcondition	Liste de projets ACTIVE non archivés, triée par score interne
Scénario nominal	1. L'utilisateur ouvre l'écran « recommandations ». 2. Le backend charge <code>profile.preferredCategories</code> et <code>profile.riskPreference</code>. 3. <code>recommendationService</code> calcule un score déterministe par projet (adéquation catégorie, cohérence avec <code>aiAnalysis.riskLevel</code>, progression, récence). 4. Les projets sont triés par score décroissant (limite paramétrable, défaut 8).
Scénarios alternatifs	1. Aucun projet ACTIVE : liste vide + message. 2. Risque ou catégorie manquants : pénalités neutres dans le score (implémentation actuelle).
Exceptions	1. Session expirée (401).

TABLE 2.5 – Description textuelle du cas « Voir les projets recommandés »

Cas d'utilisation	Signaler un contenu (projet ou commentaire)
Acteur	Utilisateur
Précondition	Utilisateur authentifié ; cible existante
Postcondition	Rapport stocké avec statut PENDING (énumération FRAUD, INAPPROPRIATE_CONTENT, SPAM, OTHER)
Scénario nominal	1. L'utilisateur ouvre le formulaire de signalement. 2. Il choisit le type, rédige la description, lie éventuellement un commentaire. 3. Le backend persiste le signalement et notifie l'administration.
Scénarios alternatifs	1. Consultation de l'historique « mes signalements ». 2. Administrateur : résolution (RESOLVED) ou classement sans suite (DISMISSED).
Exceptions	1. Champs requis manquants. 2. Signalement redondant (contraintes d'unicité <code>reports</code> côté base).

TABLE 2.6 – Description textuelle du cas « Signaler un contenu »

2.6 Conclusion

Ce chapitre a délimité le système en « boîte noire », identifié **six** acteurs du contexte (Visiteur, Utilisateur authentifié, Administrateur, passerelle de paiement démo, service d'analyse de risque orchestré, service SMTP) et structuré les besoins en **cinq** paquetages métier (Comptes, Projets, Contributions & finance, Échanges & confiance, Administration). Le tableau 2.1 fournit l'inventaire exhaustif des cas d'utilisation aligné sur l'implémentation ; les fiches 2.2–2.6 illustrent le format annexe pour des cas transverses majeurs. Le chapitre 3 traduit ces besoins en conception UML et en schémas de données.

Chapitre 3

Analyse et conception

3.1 Introduction

Ce chapitre traduit les besoins capturés au chapitre 2 en une conception UML conforme au plan (version 2). Il présente d'abord la **réalisation conceptuelle** des cas d'utilisation les plus structurants sous forme de diagrammes de séquence, en adoptant la notation attendue (acteurs, **IHM**, **contrôleur** et classes métier manipulées). Il complète chaque séquence par un diagramme de **classes participantes** (vue statique des classes mobilisées par le cas). Il dresse ensuite le **modèle du domaine** (diagramme de classes global), décrit l'**architecture du futur logiciel** et formalise les **schémas physiques des données** retenus pour MongoDB. Une conclusion synthétise les choix de conception et prépare l'implémentation.

3.2 Réalisation conceptuelle des cas d'utilisation

Les diagrammes de séquence suivants décrivent le flux d'interactions entre les acteurs et les objets du système pour un ensemble de cas d'utilisation représentatifs identifiés au chapitre 2. Chaque séquence est complétée par un diagramme de classes participantes afin d'assurer la cohérence entre la vue dynamique (interactions) et la vue statique (classes mobilisées). Ces diagrammes sont alignés sur les règles métier et sur l'implémentation livrée.

Cas retenus. Annulation de contribution, création et soumission d'un projet, investissement (paiement démo + *webhook*), validation/rejet d'un projet (administration), fourniture de coordonnées bancaires, approbation d'un retrait (administration) et mise à jour de l'état d'un projet (suspension/réactivation).

3.2.1 Annuler une contribution

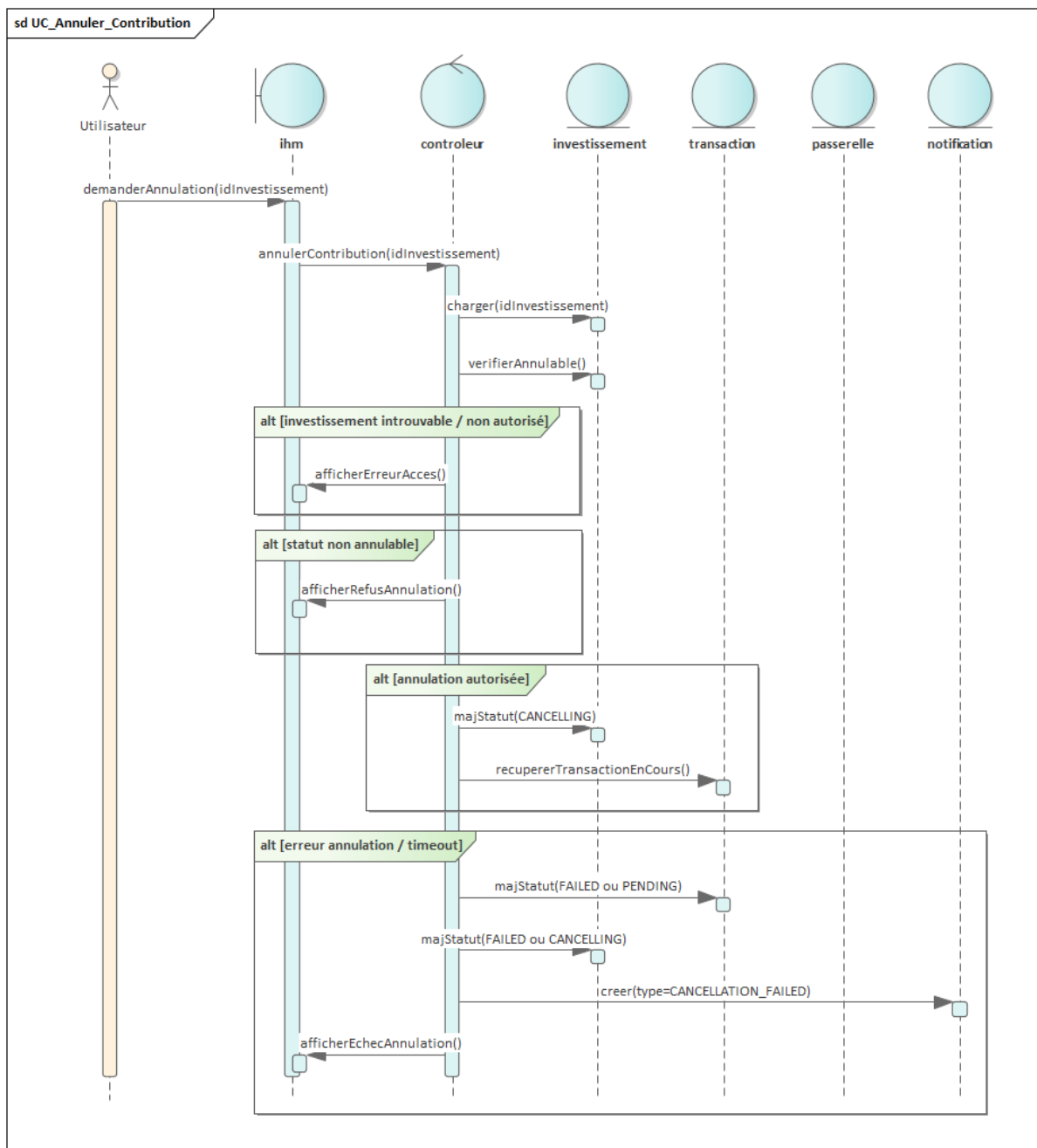


FIGURE 3.1 – Diagramme de séquence – Annuler une contribution

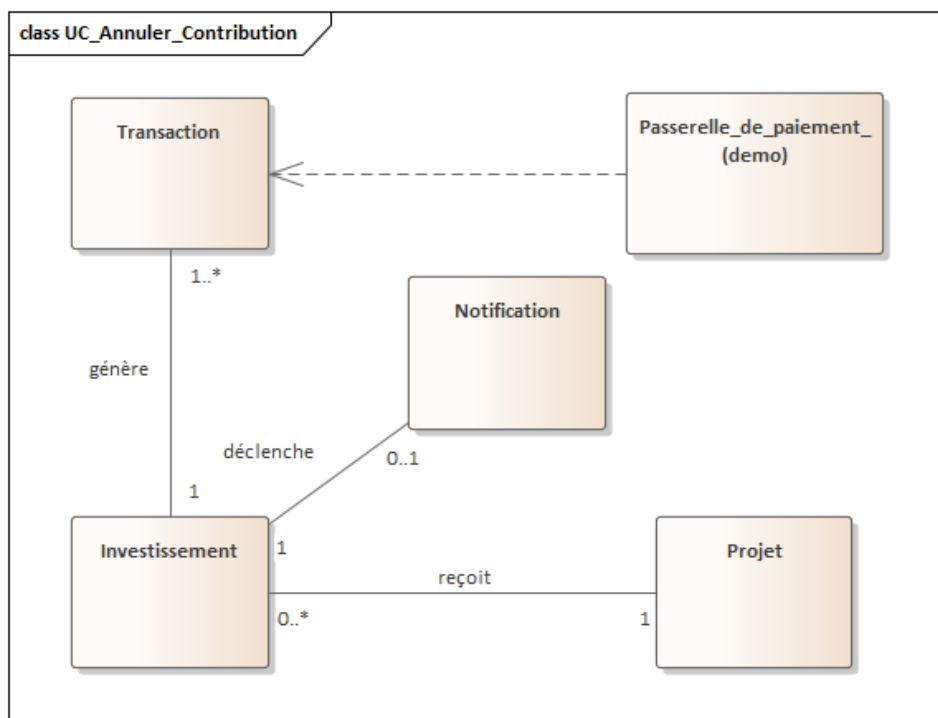


FIGURE 3.2 – Diagramme de classes participantes – Annuler une contribution

Description : L'investisseur annule une contribution tant que les conditions d'annulation sont respectées (fenêtre de grâce / paiement en cours). Le système met à jour les statuts et notifie l'utilisateur du résultat.

3.2.2 Créer et soumettre un projet

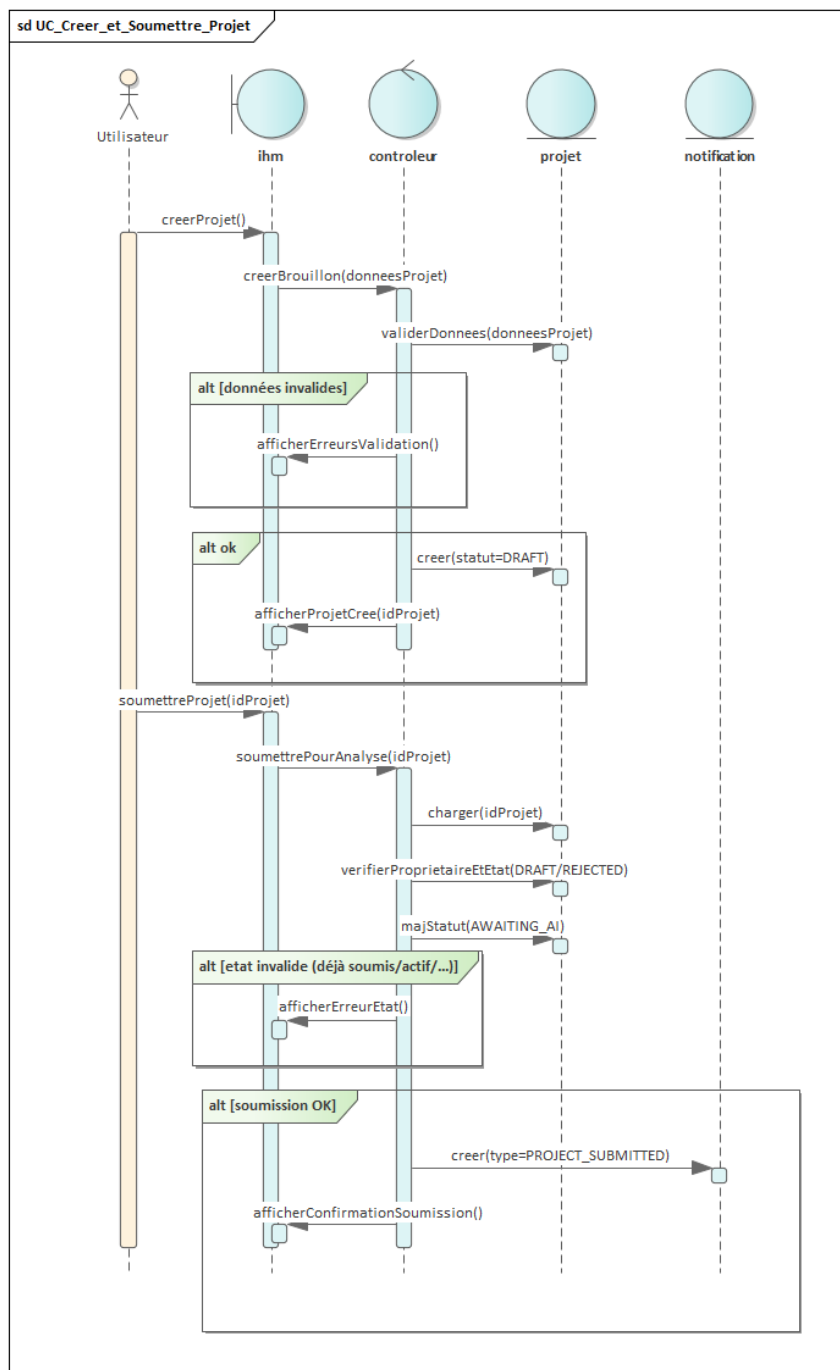


FIGURE 3.3 – Diagramme de séquence – Créer et soumettre un projet

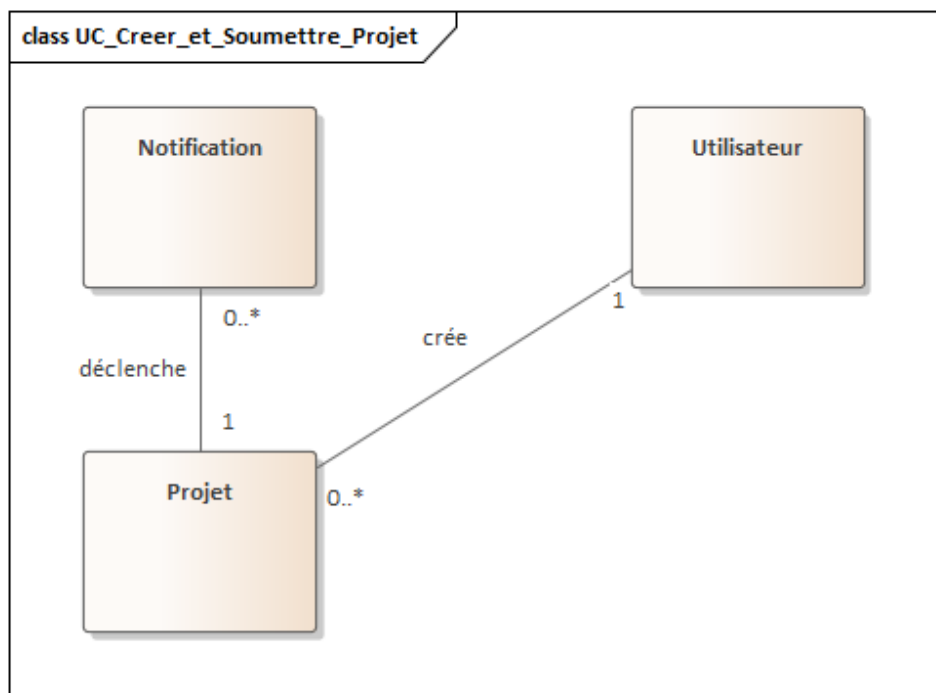


FIGURE 3.4 – Diagramme de classes participantes – Créer et soumettre un projet

Description : Le porteur de projet crée un brouillon puis le soumet. Le système passe le projet en attente d'analyse et prépare la revue administrative selon les règles métier.

3.2.3 Investir dans un projet (paiement démo + webhook)

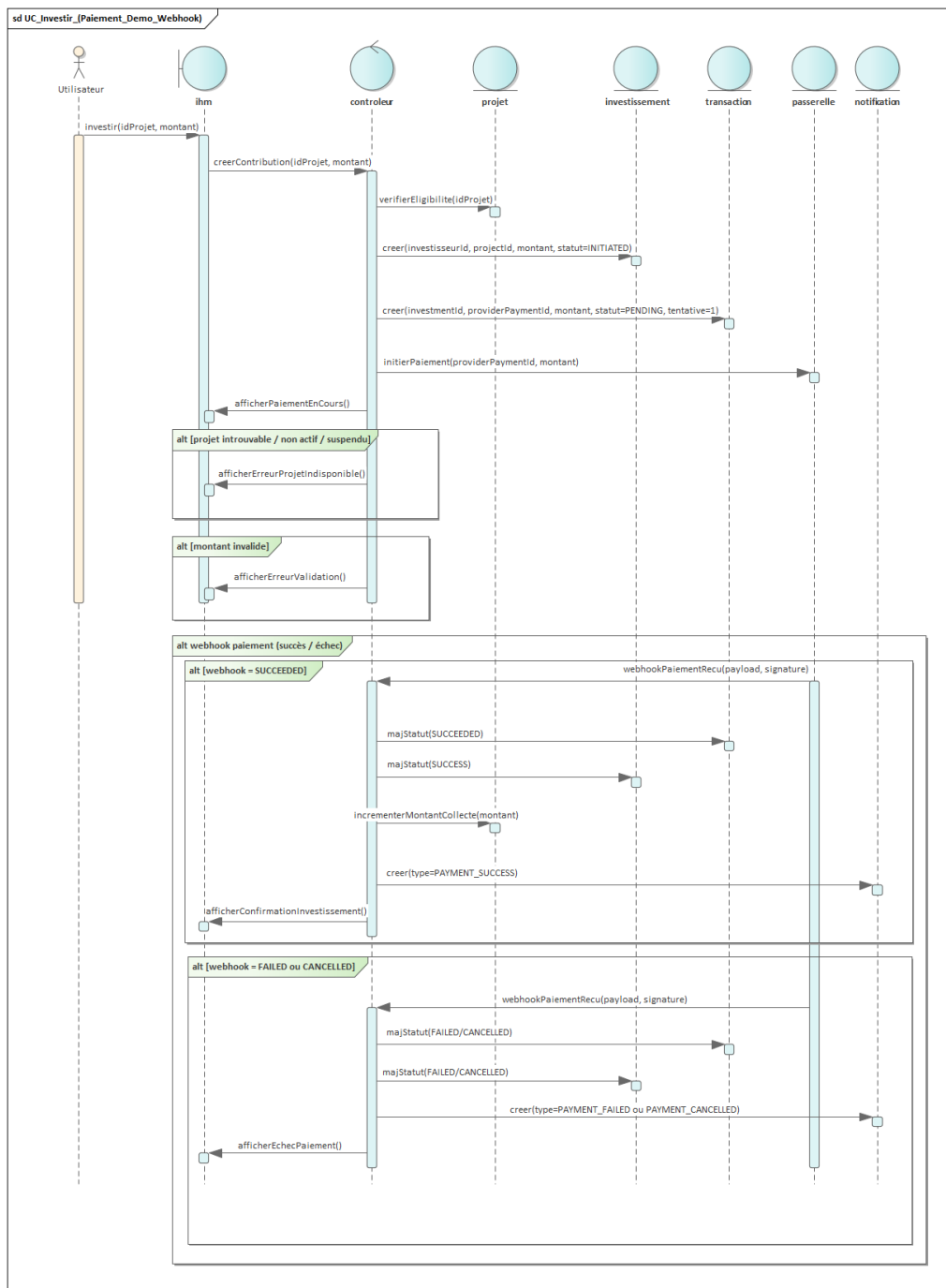


FIGURE 3.5 – Diagramme de séquence – Investir (paiement démo + webhook)

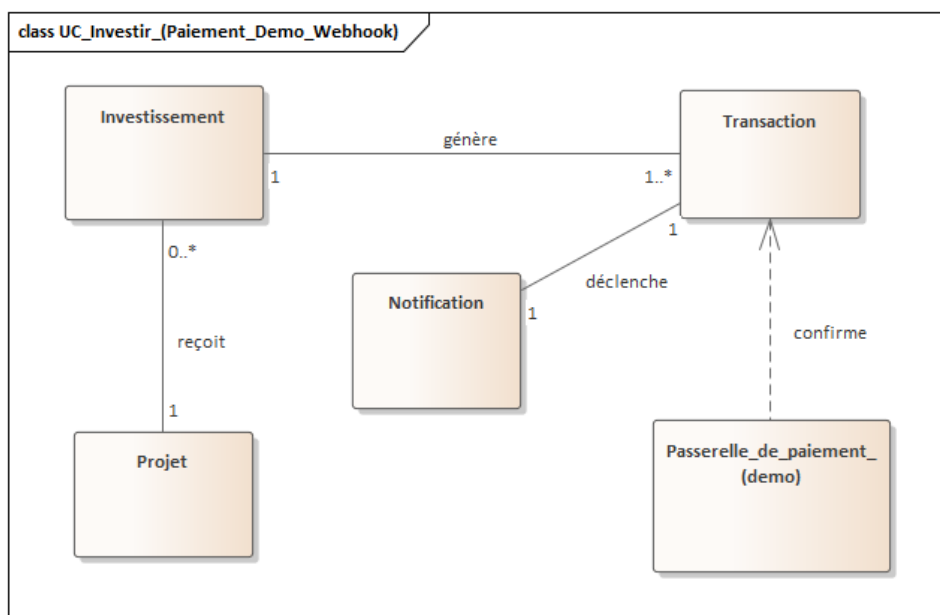


FIGURE 3.6 – Diagramme de classes participantes – Investir

Description : L'investisseur initie un paiement de démonstration. La confirmation arrive ensuite de façon asynchrone via *webhook*, ce qui met à jour la transaction et l'investissement et déclenche les notifications associées.

3.2.4 Valider un projet (Admin)

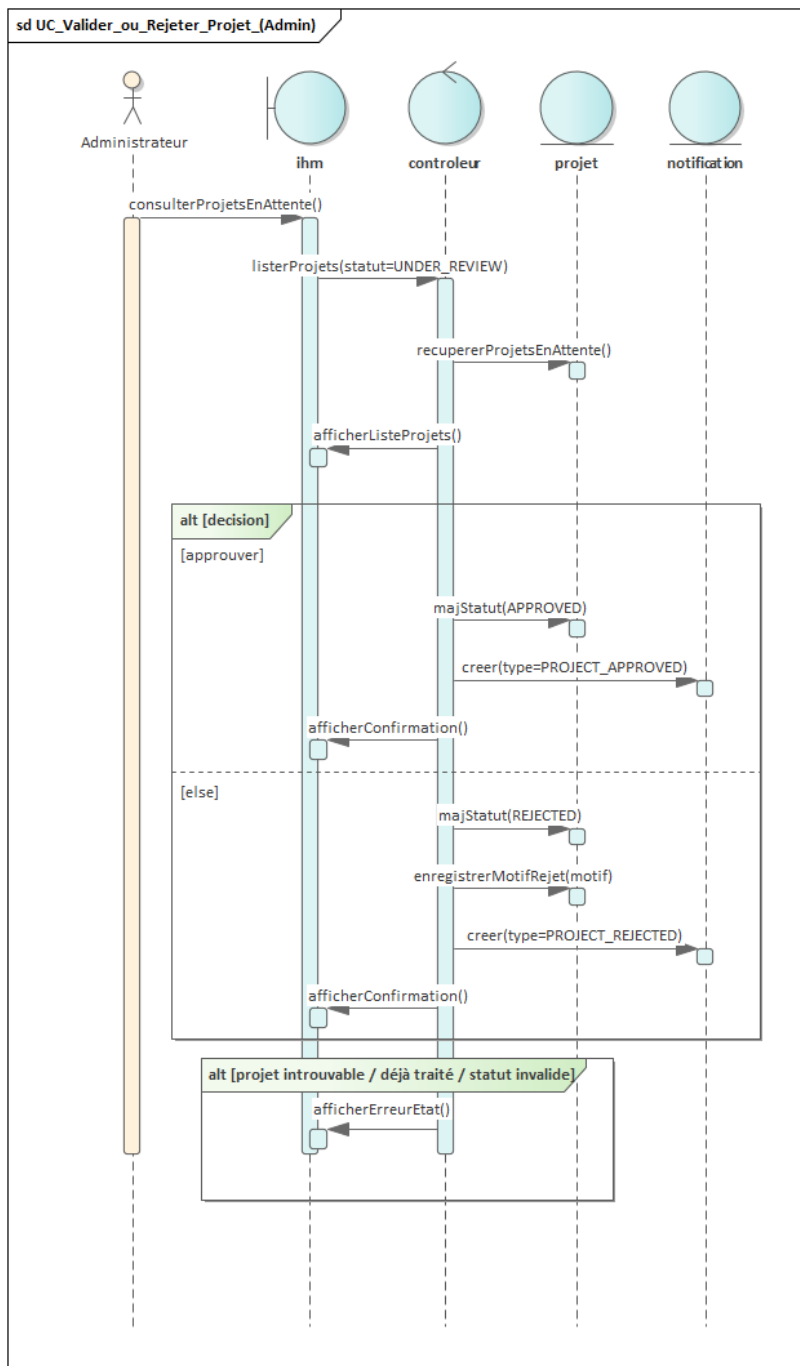


FIGURE 3.7 – Diagramme de séquence – Valider un projet

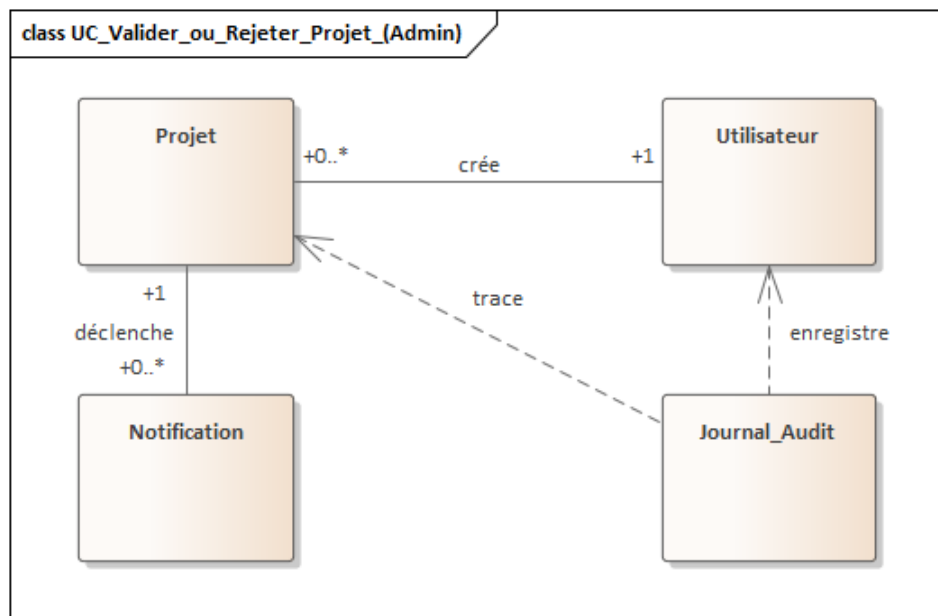


FIGURE 3.8 – Diagramme de classes participantes – Valider un projet

Description : L'administrateur examine un projet en revue et décide de l'approuver ou de le rejeter. La décision est journalisée et une notification est envoyée au créateur.

3.2.5 Fournir les coordonnées bancaires

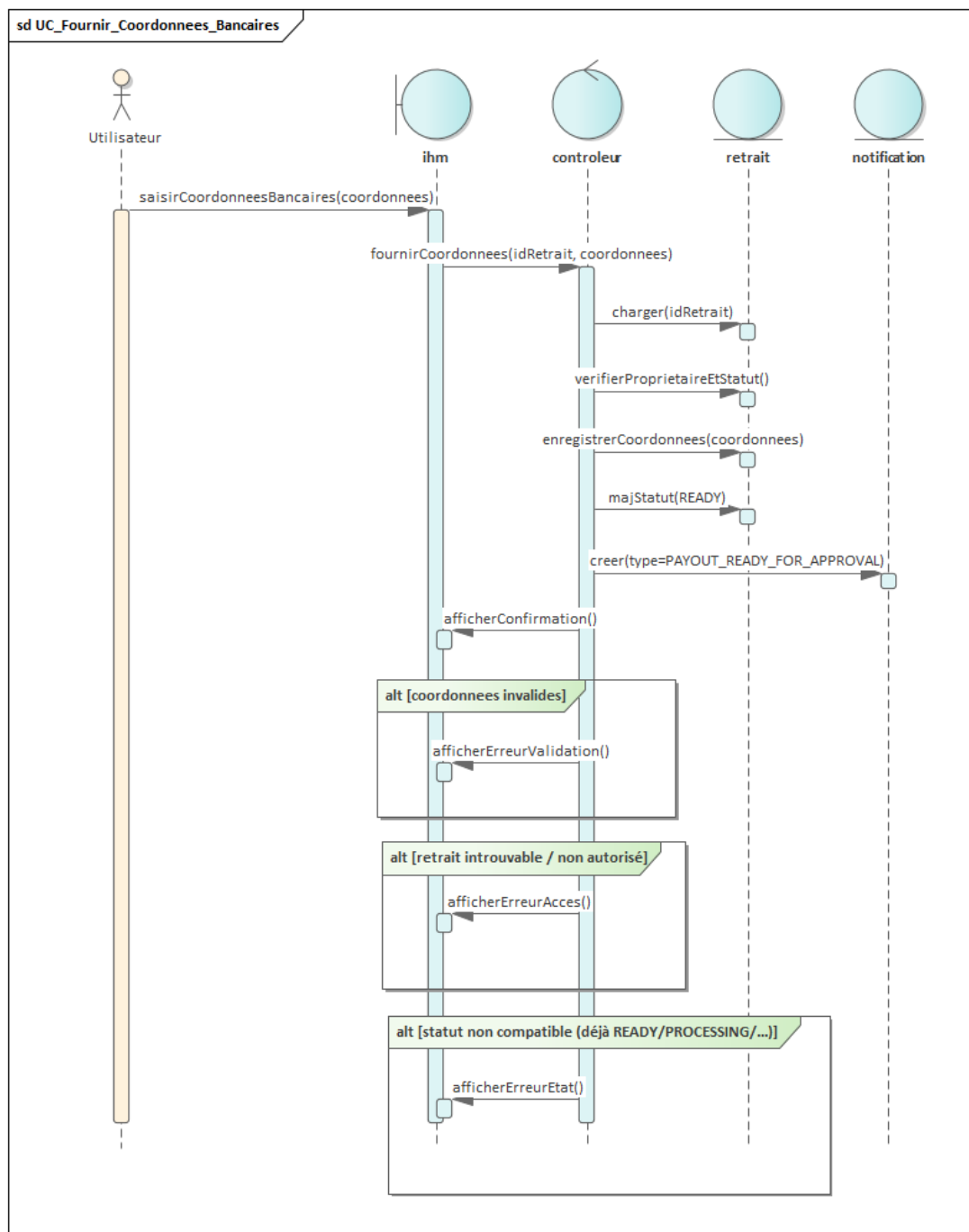


FIGURE 3.9 – Diagramme de séquence – Fournir les coordonnées bancaires

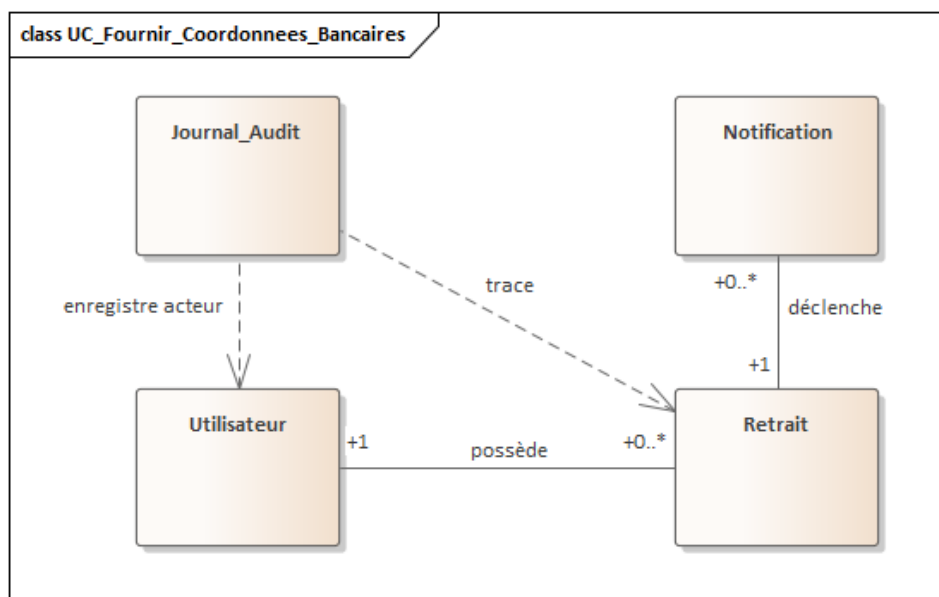


FIGURE 3.10 – Diagramme de classes participantes – Fournir les coordonnées bancaires

Description : Le créateur renseigne ses coordonnées bancaires pour un retrait en attente. Le retrait passe à l'état prêt pour validation, et l'administration est alertée.

3.2.6 Approuver un retrait (Admin)

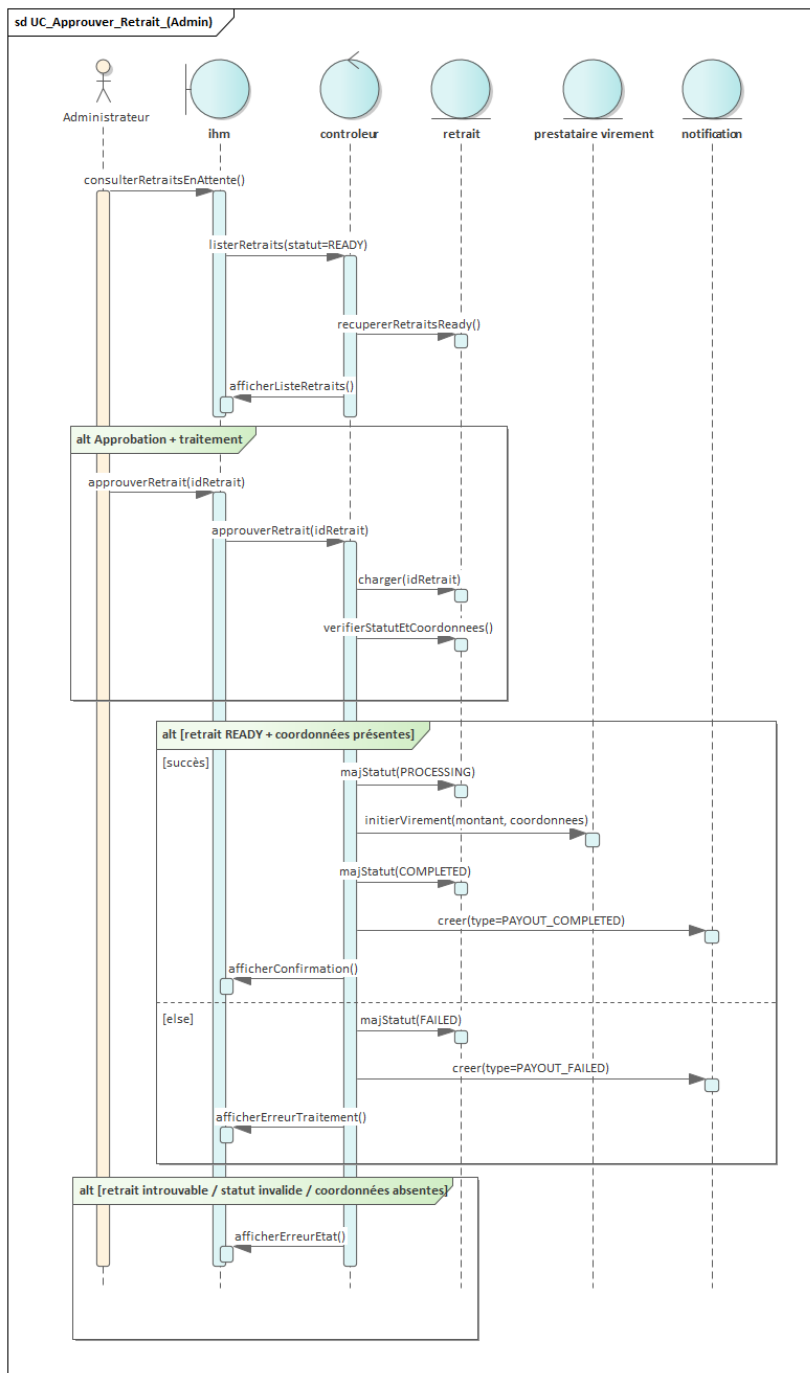


FIGURE 3.11 – Diagramme de séquence – Approuver un retrait (Admin)

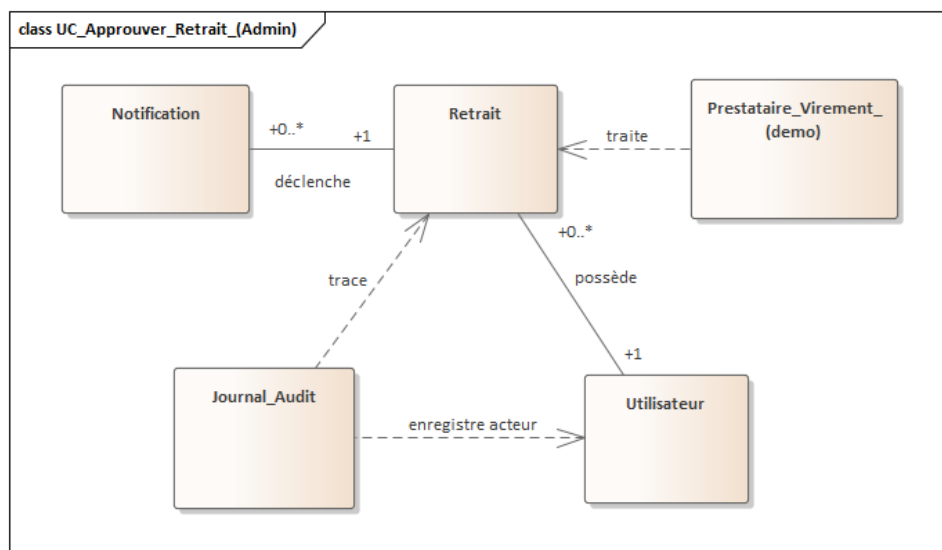


FIGURE 3.12 – Diagramme de classes participantes – Approuver un retrait

Description : L'administrateur approuve un retrait prêt à être traité. Le système initie un virement de démonstration, met à jour l'état du retrait, journalise l'action et notifie le créateur.

3.2.7 Mise à jour de l'état d'un projet (Admin)

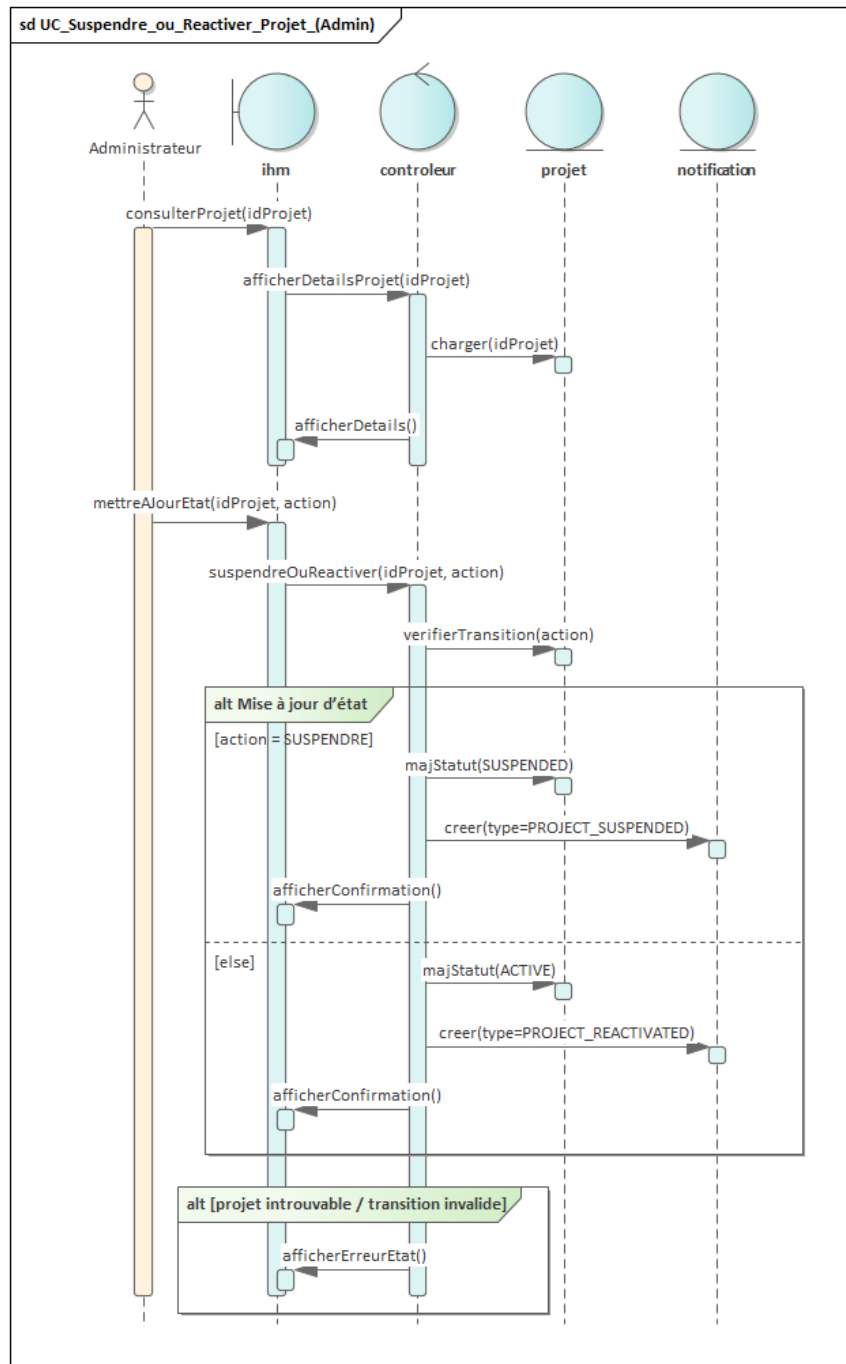


FIGURE 3.13 – Diagramme de séquence – Mise à jour de l'état d'un projet (suspension / réactivation)

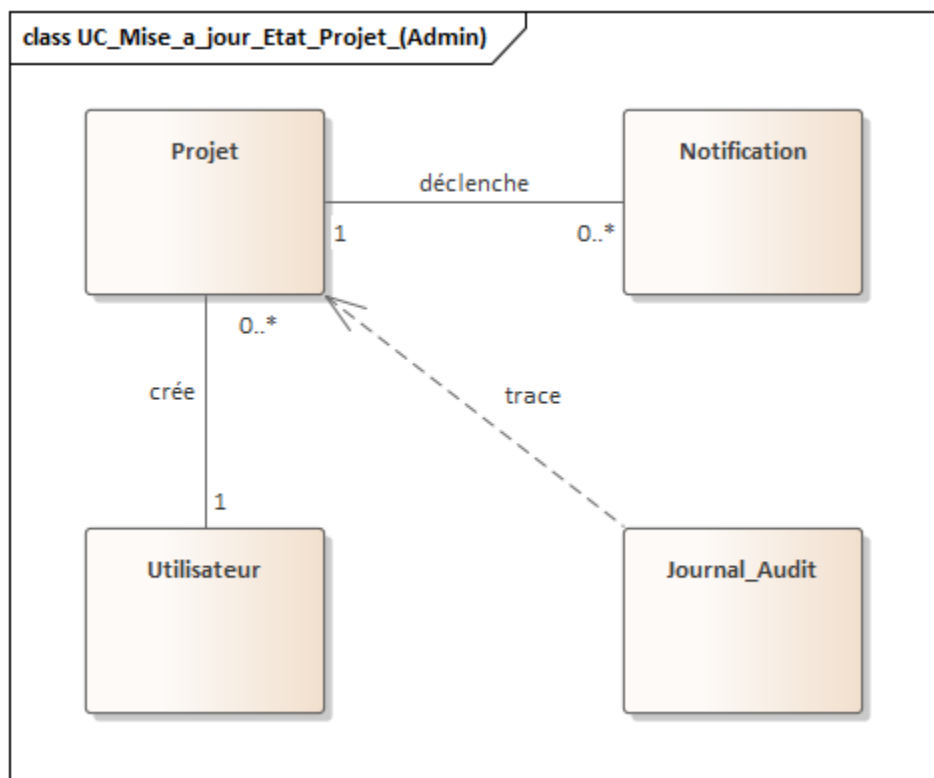
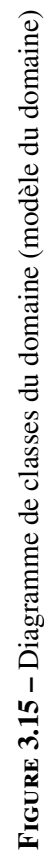


FIGURE 3.14 – Diagramme de classes participantes – Mise à jour de l'état d'un projet

Description : L'administrateur met à jour l'état d'un projet (suspension ou réactivation) selon les règles de cycle de vie. L'opération est tracée et une notification est envoyée au créateur.

3.3 Modèle du domaine

Lecture du diagramme. Le diagramme de classes ci-dessus constitue le **modèle du domaine** de la plateforme. Il présente les principales entités métier (**Utilisateur, Projet, Investissement, Transaction, Retrait, Notification, Commentaire, Signalement**) ainsi que leurs attributs, opérations et relations (avec multiplicités). Les énumérations (rôle, statuts, types de notification) rendent explicites les cycles de vie et les règles de gestion. Les contraintes fondamentales y sont résumées : (i) unicité de l'e-mail, (ii) la collecte d'un projet ne dépasse pas l'objectif de financement, et (iii) au plus un retrait par projet. Ce modèle sert de référence aux sections suivantes (architecture et schémas physiques MongoDB).



3.4 Architecture du futur logiciel

3.4.1 Vue d'ensemble en couches

L'application suit une architecture client-serveur à trois couches, déployée sur un cloud commercial :

- **Couche présentation** : application React (SPA) servie via Vite, communiquant avec le backend par appels REST sécurisés (cookies HttpOnly).
- **Couche métier** : Node.js + Express, qui expose les points d'accès API, applique les règles de validation (Zod), orchestre les services internes et gère les transactions MongoDB.
- **Couche données** : MongoDB (replica set requis pour les transactions multi-documents).

Les services internes (`projectService`, `investmentService`, `payoutService`, `adminProjectService`, `adminUserService`, `notificationService`, `recommendationService`, `reportService`, `authService`, `workflowInternalService`, `geminRiskService`, `geminChatService`) sont des modules distincts au sein du backend. Ils sont assemblés par l'*Express router* et leurs traitements asynchrones sont délégués à des files BullMQ/Redis (e-mails, retentatives) ou à des workflows n8n (analyse de risque); le chatbot est traité par l'API Express (POST `/api/projects/:id/chat`) avec appel LLM côté serveur.

3.4.2 Intégration de n8n

n8n est un outil d'orchestration visuelle de workflows utilisé pour découpler la logique métier des services externes. Dans le projet :

- **Analyse de risque** : à la soumission d'un projet, le backend déclenche un workflow n8n qui interroge le service LLM, reçoit un rapport structuré, applique le seuil d'auto-rejet ($\text{gap} \geq 30\%$), puis met à jour le projet et notifie les acteurs concernés.
- **Chatbot** : un workflow n8n centralise les prompts et la mémoire conversationnelle pour répondre aux questions des utilisateurs sur un projet public.

Avantages obtenus : **découplage** (le backend ne connaît qu'un endpoint webhook + signature), **résilience** (retries exponentiels 1/2/4 minutes, max 3 tentatives, puis stockage en `FailedWorkflowEvents`) et **évolutivité** (de nouveaux workflows peuvent être ajoutés sans modifier le code applicatif).

3.4.3 Composants asynchrones (BullMQ / Redis)

Les e-mails critiques (validation, contribution, paiement, retrait, désactivation) et les opérations de reprise sont gérés par des files BullMQ persistées dans Redis. Cette architecture absorbe les pics de charge et fournit un mécanisme uniforme de relance et de DLQ (FailedRefund, FailedPayoutEvents, FailedCancellationEvents). Les jobs récurrents (cron) sont gérés par le service interne cronService : expiration des projets et investissements, reprise des analyses bloquées, nettoyage des sessions et des jetons de rafraîchissement.

3.4.4 Vue logique en paquetages

Le diagramme de paquetages de la figure 3.16 décompose l'application en domaines fonctionnels cohérents (*Comptes, Projets, Contributions & finance, Échanges & modération, Administration*) et matérialise les dépendances vers les composants externes (*n8n, BullMQ/Redis, MongoDB*). Chaque paquetage regroupe les modèles, services et routes qui implémentent une responsabilité métier identifiée au chapitre 2.

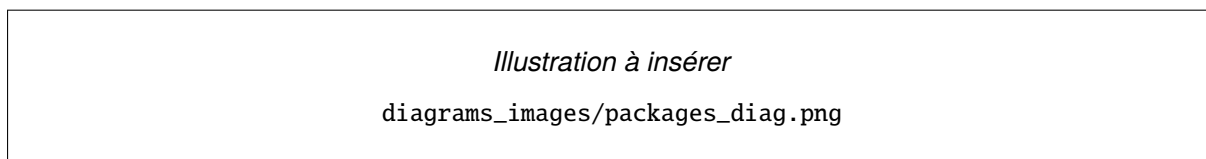


FIGURE 3.16 – Diagramme de paquetages de l'application

3.4.5 Vue physique

La figure 3.17 synthétise l'architecture déployée : application front-end React, back-end Node.js/Express, base MongoDB, broker Redis pour BullMQ, instance n8n pour l'orchestration et service LLM externe.

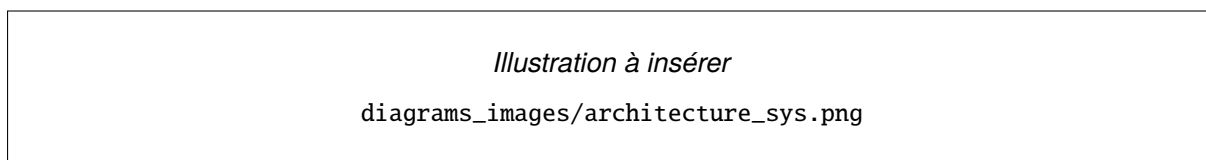


FIGURE 3.17 – Architecture générale de l'application

3.4.6 Décisions de conception structurantes

Les décisions suivantes ont été prises pour satisfaire les exigences non fonctionnelles (sécurité, résilience, traçabilité) :

- **Authentification par JWT (cookies HttpOnly)** : access token court + refresh token à plus longue durée, tous deux servis en cookies HttpOnly; Secure; SameSite, afin d'éviter l'exposition aux scripts client et au CSRF.
- **Base de données NoSQL document** : MongoDB choisi pour la flexibilité du document Project (champs variables, sous-document aiAnalysis) et la simplicité d'évolution du schéma.
- **Analyse IA orchestrée** : déclenchée par n8n, avec timeouts, tentatives et auto-rejet contrôlé ($\text{gap} \geq 30 \%$).
- **Païement asynchrone idempotent** : la confirmation arrive par *webhook* signé; la combinaison (provider, providerPaymentId) est unique pour absorber les tentatives du fournisseur.
- **Mise à jour conditionnelle atomique** du financement : updateOne sur projects avec contrainte $\text{currentFunding} + \text{amount} \leq \text{fundingGoal}$ pour empêcher tout sur-financement, même en concurrence.
- **Files BullMQ + DLQ** : tentatives exponentielles, stockage des échecs définitifs dans des collections dédiées pour reprise manuelle par l'administrateur.
- **Chiffrement des coordonnées bancaires** : algorithme AES-256-GCM avec IV par enregistrement et clé maître chargée hors code, pour respecter les exigences de confidentialité des RIB.
- **Limitation de débit (rate limiting)** : en sortie API publique, plus stricte sur les surfaces sensibles (commentaires, signalements, authentification) pour limiter les abus.

3.5 Schémas physiques des données

3.5.1 Schéma physique (MongoDB)

Le schéma physique est dérivé du modèle du domaine et des schémas Mongoose. Comme la solution repose sur une base **NoSQL** (MongoDB), la structure est présentée sous forme de **collections** et de **références** (*ObjectId*) plutôt que sous forme de tables relationnelles. Dans ce rapport, la description est fournie **sans figures** afin d'éviter toute dépendance à des exports/outils

externes.

- `users` : stocke les identifiants, le rôle, le statut actif, les jetons de réinitialisation, et le profil (préférences de risque, catégories).
- `projects` : contient les détails du projet ainsi que l'analyse IA embarquée (`aiAnalysis`).
- `investments` : lie un investisseur à un projet avec un montant et un statut.
- `transactions` : enregistre les informations de paiement (démonstration / extensible).
- `payouts` : stocke les retraits (statut, coordonnées bancaires chiffrées, suivi du virement).
- `comments` : stocke les commentaires (modération, masquage, suppression).
- `reports` : gère les signalements.
- `notifications` : stocke les notifications in-app.
- `auditLogs` : journalise les actions sensibles (traçabilité).
- `failedWorkflowEvents` : stocke les échecs des workflows pour reprise manuelle.
- `failedCancellationEvents` : stocke les échecs d'annulation de paiement lors de la désactivation d'un utilisateur.
- `failedRefundEvents` : stocke les échecs de remboursement (sur-financement, expiration, etc.).
- `failedPayoutEvents` : stocke les échecs de traitement des retraits.

3.5.2 Justification du modèle NoSQL

Le choix de MongoDB repose sur la nature semi-structurée des données. Les projets contiennent des champs variables (images, documents, analyse IA) qui évoluent au fil du temps. Un modèle relationnel imposerait des tables supplémentaires ou des colonnes redondantes, tandis que MongoDB permet d'ajouter des champs sans migration. De plus, la possibilité d'utiliser des sous-documents (embedding) pour `profile` et `aiAnalysis` améliore les performances de lecture.

3.5.3 Indexation et relations

Pour optimiser les performances, les index suivants sont créés :

- `users.email` : unique.
- `projects.status` et `projects.createdAt` : pour filtrer les projets actifs.
- `projects.category` : pour les recommandations.
- `investments.investorId` et `investments.projectId` : pour retrouver rapidement les investissements d'un utilisateur ou d'un projet.

- `transactions.investmentId` : index simple (plusieurs transactions peuvent exister pour un même investissement, par exemple en cas de tentative).
- `transactions.provider` et `transactions.providerPaymentId` : index unique pour assurer l'idempotence des webhooks.
- `notifications.userId` et `notifications.read` : pour afficher les notifications non lues.
- `failedWorkflowEvents.resolved` et `failedWorkflowEvents.createdAt` : pour lister les échecs non résolus.
- `failedCancellationEvents.resolved` et `failedCancellationEvents.createdAt` : idem.

Les relations sont gérées par références (`ObjectId`). L'embedding est utilisé pour les profils utilisateur et l'analyse IA dans les projets. En MongoDB, les jointures sont effectuées au niveau de l'application en utilisant les méthodes `populate` (avec Mongoose) ou des requêtes manuelles.

3.5.4 Dictionnaire des données

Le tableau 3.1 présente une description détaillée des principaux champs des collections.

Collection	Champ	Description
users	email passwordHash role isActive resetToken resetTokenExpiry profile.firstName profile.lastName profile.riskPreference profile.preferredCategories	Adresse email de l'utilisateur (unique). Mot de passe haché avec bcrypt. Rôle de l'utilisateur (USER ou ADMIN). Statut du compte (actif / désactivé). Jeton de réinitialisation du mot de passe. Date d'expiration du jeton. Prénom. Nom de famille. Tolérance au risque (LOW, MEDIUM, HIGH). Catégories de projets préférées.
projects	title description category fundingGoal currentFunding status aiStatus aiAnalysis.report aiAnalysis.riskLevel	Titre du projet. Description détaillée. Catégorie (ex : Technologie, Art, etc.). Montant cible. Montant collecté (mise à jour atomique conditionnelle). État du projet (DRAFT, AWAITING_AI, UNDER_REVIEW, APPROVED, ACTIVE, REJECTED, FUNDED, CLOSED, SUSPENDED). État de l'analyse IA (PENDING, COMPLETED, FAILED). Rapport structuré (résumé, avantages, inconvénients, améliorations, questions, score). Niveau de risque (LOW, MEDIUM, HIGH).
investments	investorId projectId amount status updatedAt	Référence vers l'utilisateur investisseur. Référence vers le projet. Montant investi (strictement positif). État de l'investissement (INITIATED, SUCCESS, FAILED, CANCELLING, CANCELLED, REFUNDED). Date de dernière mise à jour (utilisée pour repérer les contributions restées en INITIATED trop longtemps).
transactions	investmentId provider providerPaymentId status	Référence vers l'investissement. Fournisseur de paiement (mock en démonstration). Identifiant du paiement chez le fournisseur (index unique avec provider pour l'idempotence). État de la transaction (PENDING, SUCCEEDED, FAILED, REFUNDED, CANCELLED).
reports	reporterId projectId type status	Référence vers l'utilisateur ayant signalé. Référence vers le projet signalé. Type de signalement (FRAUD, SPAM, etc.). État du signalement (PENDING, RESOLVED).
notifications	userId type read	Référence vers l'utilisateur destinataire. Type de notification (ex : PROJECT_APPROVED). Indicateur de lecture.
failedWorkflowEvents	workflowType payload error	Type de workflow (risk-analysis, notification). Données initiales du workflow. Message d'erreur.

3.6 Conclusion

Ce chapitre a traduit les besoins métier en une conception UML cohérente : les diagrammes de séquence formalisent le déroulement des cas d'utilisation critiques, le diagramme de classes capture la structure du domaine, et l'architecture en couches précise la place des composants asynchrones (n8n, BullMQ/Redis) et les décisions structurantes (JWT en cookies, idempotence des *webhooks*, atomicité des mises à jour de financement, AES-256-GCM pour les RIB, files DLQ pour la reprise des échecs). Les schémas physiques MongoDB et le dictionnaire des données fournissent le socle directement exploitable par l'implémentation décrite au chapitre suivant.

Chapitre 4

Implémentation et tests

4.1 Introduction

Ce chapitre décrit la réalisation technique de la plateforme. Il présente d'abord l'environnement de réalisation (matériel et logiciel) ainsi que les choix technologiques retenus, puis détaille la mise en œuvre des cas d'utilisation les plus structurants (authentification, projets, paiement simulé, recommandation, notifications), la stratégie de test et le scénario de déploiement. Les écrans représentatifs de l'application sont inclus avec leur libellé final ; les diagrammes de séquence d'analyse présentés au chapitre 3 ont guidé l'implémentation.

4.2 Environnement de réalisation

4.2.1 Environnement matériel

Le développement a été réalisé sur un ordinateur portable avec les caractéristiques suivantes :

- Processeur : Intel Core i5-10300H @ 2.50 GHz.
- Mémoire RAM : 16 Go.
- Stockage : SSD 512 Go.

4.2.2 Environnement logiciel

Les principaux outils et bibliothèques utilisés sont les suivants :

- **Backend** : Node.js (v20.11.1), Express (v4.19.2), Mongoose (v8.2) pour l'accès à MongoDB, Zod pour la validation, BullMQ + ioredis pour les files asynchrones, Nodemailer pour l'envoi d'e-mails, jsonwebtoken (v9.0.2) et bcrypt (v5.1.1) pour la sécurité.
- **Frontend** : React (v18.3.1) avec Vite, Axios pour les appels API (avec intercepteurs de rafraîchissement de session), Tailwind CSS pour le style.
- **Base de données** : MongoDB 6+ (replica set requis pour les transactions multi-documents)
— déploiement local et instance managée utilisée pour la démonstration.
- **Orchestration et IA** : n8n (v1.40.0) pour les workflows d'analyse de risque ; service LLM sollicité par n8n pour l'analyse et directement par le backend pour le chatbot (/api/projects/:id/chat), avec timeouts et tentatives.
- **Outils** : Visual Studio Code, Git/GitHub pour le versionnage, Enterprise Architect 15 pour la modélisation UML, Postman pour le test manuel des API.
- **Tests** : scripts E2E Node.js (e2e-lifecycle-payout-api.js, e2e-lifecycle-scenarios.js, e2e-edgecases-api.js, e2e-interactions.js) + tests manuels guidés sur l'interface.

4.2.3 Synthèse des choix technologiques

Le tableau 4.1 synthétise les technologies retenues et la justification de chaque choix.

Composant	Technologie	Justification
Frontend	React.js + Vite	Composants réutilisables, écosystème riche, démarrage instantané.
Backend	Node.js + Express	Environnement JavaScript unifié, non bloquant, adapté aux API REST et aux traitements asynchrones.
Validation	Zod	Schémas typés en TypeScript, messages d'erreur clairs côté API.
Base de données	MongoDB	Modèle document, sous-documents pour <i>aiAnalysis</i> , transactions multi-documents pour la cohérence.
Files asynchrones	BullMQ + Redis	Retentatives exponentielles, DLQ, supervision des jobs (e-mails, reprises).
Authentification	JWT (cookies HttpOnly)	Sans état, accès court / refresh long, protection contre l'XSS.
Chiffrement RIB	AES-256-GCM	Confidentialité des coordonnées bancaires, IV unique par enregistrement.
IA (analyse / chatbot)	Service LLM + n8n (analyse); Express (chatbot)	Analyse orchestrée, tentatives, auto-rejet sur incohérence budgétaire ($\geq 30\%$), heuristiques explicables; chatbot via <code>/api/projects/:id/chat</code> .
Orchestration	n8n	Découplage des workflows d'analyse; évolutions sans toucher au cœur Express pour ces flux.
Hébergement (cible)	Hébergement cloud managé (frontend, backend, base)	Déploiement continu, certificat SSL, sauvegardes managées.

TABLE 4.1 – Technologies utilisées

4.3 Réalisation technique des cas d'utilisation

Cette section décrit la mise en œuvre des principales fonctionnalités, en s'appuyant sur les diagrammes de séquence d'analyse pour guider les interactions entre les composants techniques.

4.3.1 Authentification et gestion des utilisateurs

Description fonctionnelle

L'authentification repose sur des **JWT stockés dans des cookies HttpOnly** : un jeton d'accès (*access token*) et un jeton de rafraîchissement (*refresh token*). À la connexion, le serveur vérifie

les identifiants et dépose ces deux jetons dans des cookies (HttpOnly, SameSite=Lax, Secure en production). Le jeton de rafraîchissement permet de prolonger la session sans ressaisir le mot de passe, via l'endpoint de rafraîchissement.

Les mots de passe sont hachés avec bcrypt. Le schéma utilisateur inclut un champ **isActive** (booléen) qui est initialement **false** à l'inscription. Un e-mail de vérification est envoyé avec un **code OTP** (et un lien en secours). Le compte est activé après saisie du code (ou clic du lien). L'administrateur peut également désactiver ou réactiver un compte. Les routes protégées utilisent des middlewares d'authentification basés sur des cookies HttpOnly (access/refresh) et un contrôle de rôle pour l'administration. Sur le frontend, un intercepteur Axios surveille les réponses 401 (jeton expiré) ; il appelle alors `/api/auth/refresh` et rejoue la requête originale.

Bibliothèques utilisées

- `jsonwebtoken` — émission et vérification des jetons d'accès et de rafraîchissement.
- `bcrypt` — hachage des mots de passe (salt rounds ≥ 10).
- `cookie-parser` — lecture des cookies HttpOnly côté Express.
- `nodemailer` (combiné à `bullmq`) — envoi des e-mails OTP et de notifications de sécurité.
- `zod` — validation stricte des entrées (e-mail, mot de passe, OTP).
- `axios` (frontend) — appels API et intercepteur de rafraîchissement automatique.

Capture VSCode à insérer : intercepteur Axios de rafraîchissement

À faire : insérer une capture d'écran VSCode montrant l'intercepteur Axios qui gère une réponse 401 en appelant `/api/auth/refresh`, puis en rejouant la requête initiale.

Captures d'écran

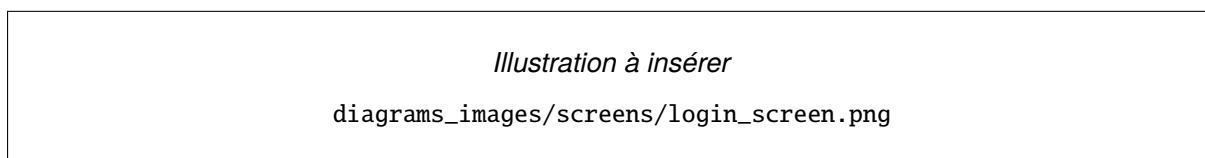


FIGURE 4.1 – Page de connexion

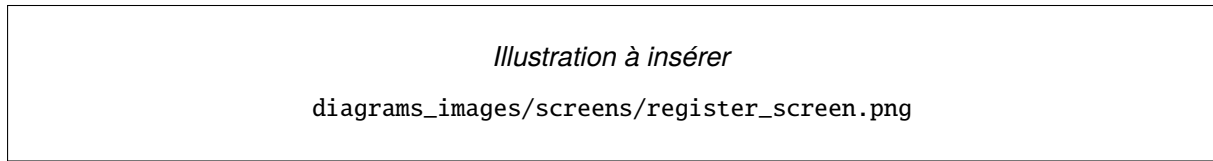


FIGURE 4.2 – Formulaire d’inscription avec saisie du code OTP

Composants techniques associés

Cron de nettoyage des jetons de rafraîchissement expirés (`cronService`), table d’audit des tentatives de connexion, blocage progressif (*rate limiting* via `express-rate-limit`) sur `/api/auth/login` et `/api/auth/forgot-password`.

4.3.2 Gestion des projets

Description fonctionnelle

La création et la mise en ligne d’un projet suivent un cycle métier explicite :

1. Validation des données entrantes (champs obligatoires, type de données).
2. Création du projet en brouillon (DRAFT) par le porteur de projet.
3. Soumission explicite à l’analyse de risque : passage en `AWAITING_AI` et mise en file du workflow.

Le workflow `n8n` (voir figure ??) exécute les actions suivantes :

1. Appel au service LLM avec un prompt contenant les informations du projet.
2. Extraction du score de risque de la réponse.
3. Mise à jour du projet dans MongoDB avec le score et le niveau de risque, puis passage en `UNDER_REVIEW` (revue humaine).
4. Notification de l’administration (in-app + e-mail selon la politique).

En cas d’indisponibilité temporaire du service IA (`quota/timeout/503`), le projet reste en attente (`AWAITING_AI`) et le système planifie des tentatives automatiques. Un administrateur peut également déclencher une relance manuelle via l’interface d’administration (`/api/admin/projects/:id/retry-ai`).

Bibliothèques utilisées

- `mongoose` — modélisation des collections `projects` et `users`, transitions de statut, index secondaires.

- zod — validation des champs *title, description, category, fundingGoal, deadline*.
- multer — réception du média et de la pièce jointe du projet.
- axios (côté backend) — appel signé vers le *webhook* n8n d'analyse de risque.
- n8n — orchestration du workflow d'analyse IA + appel au service LLM.

Captures à insérer : workflow n8n (analyse de risque)

À faire : insérer une ou deux captures de l'interface n8n montrant les nœuds principaux du workflow (réception de la demande, appel LLM, validation/formatage de la réponse, mise à jour du projet, notification / gestion des erreurs).

Captures d'écran

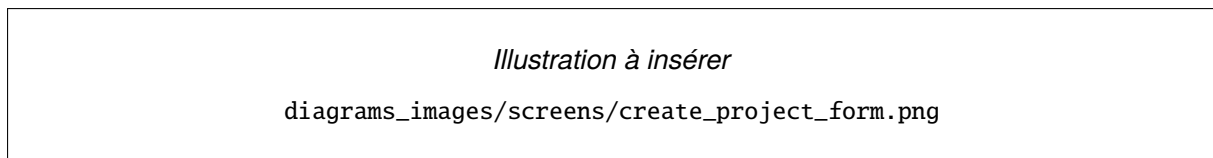


FIGURE 4.3 – Formulaire de création de projet (vue porteur)

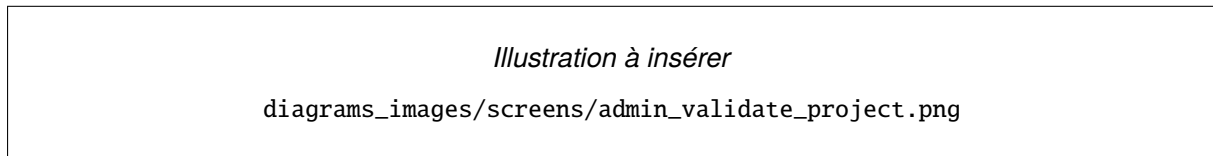


FIGURE 4.4 – Console d'administration : validation/rejet d'un projet en UNDER_REVIEW

Composants techniques associés

Webhook signé HMAC entre le backend et n8n, file BullMQ pour les tentatives d'analyse IA, cron `retryStuckAiAnalyses` pour les projets bloqués en `AWAITING_AI`, DLQ `failedWorkflowEvents` pour les échecs définitifs, journal d'audit des transitions de statut.

4.3.3 Parcours de paiement (simulation)

Description fonctionnelle

Le parcours de paiement est simulé (mock) afin de valider la logique métier (statuts, webhooks signés, idempotence) sans dépendre d'un prestataire externe.

- **Création du lien de paiement** : lorsqu'un utilisateur initie un soutien, le backend retourne une URL de paiement simulé. L'investissement et la transaction sont créés en INITIATED / PENDING.
- **Webhook mock signé** : une confirmation HTTP signée met à jour transaction/investissement et ajoute le montant au `currentFunding` du projet, sous une condition atomique qui empêche de dépasser l'objectif de financement.
- **Coordonnées bancaires (RIB)** : les RIB des porteurs de projet sont chiffrés en AES-256-GCM (IV unique par enregistrement) avant stockage et utilisés au moment du *payout*.

Bibliothèques utilisées

- `mongoose` — collections `investments`, `transactions`, `payouts`, transactions multi-documents (*replica set*).
- `crypto` (Node.js) — vérification HMAC du *webhook* et chiffrement AES-256-GCM des RIB.
- `bullmq` et `ioredis` — file de tentatives pour les remboursements et les paiements bloqués.
- `zod` — validation du *payload* de *webhook* et des saisies bancaires.

Capture VSCode à insérer : mise à jour atomique du financement

À faire : insérer une capture d'écran VSCode montrant la mise à jour atomique qui empêche `currentFunding` de dépasser `fundingGoal` lors d'un *webhook* concurrent. Cette approche garantit l'intégrité des données même en cas de *webhooks* concurrents.

Captures d'écran

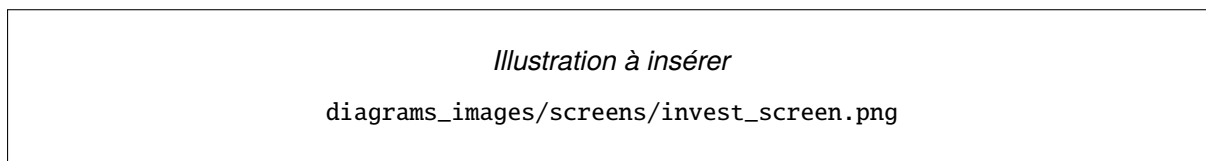


FIGURE 4.5 – Formulaire de soutien : saisie du montant et lancement du paiement

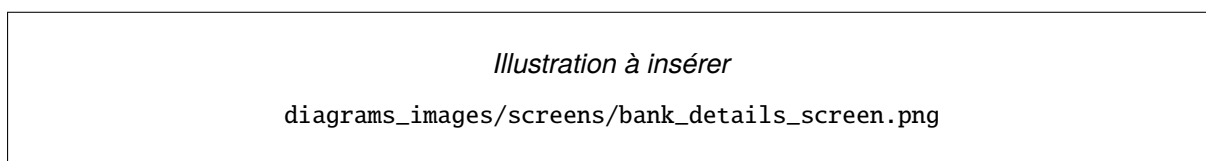


FIGURE 4.6 – Saisie des coordonnées bancaires (chiffrement AES-256-GCM avant stockage)

Composants techniques associés

Webhook HMAC idempotent (clé unique `provider+providerPaymentId`), file BullMQ `refund-queue` pour les remboursements, cron de retentative des *payouts* en échec, DLQ `failedRefund` et `failedPayoutEvents` pour reprise manuelle par l'administrateur.

4.3.4 Nettoyage des investissements abandonnés

Description fonctionnelle

Un job planifié (cron interne) exécute régulièrement un nettoyage des investissements restés au statut `INITIATED` au-delà d'un seuil (par défaut 30 minutes), en s'appuyant sur l'horodatage `updatedAt` (ou `createdAt`) de la collection `investments`. Ces investissements sont automatiquement marqués `FAILED`, la transaction associée est mise à jour, et une notification est envoyée à l'investisseur. Ce mécanisme évite l'accumulation d'enregistrements orphelins et améliore l'expérience utilisateur.

Bibliothèques utilisées

- `node-cron` — planification du job d'expiration.
- `mongoose` — requêtes ciblées sur `status` : `INITIATED` et ancienneté (`updatedAt/createdAt`).
- `bullmq` — production d'un événement de notification consommé en aval.

Capture VSCode à insérer : nettoyage périodique des investissements abandonnés

À faire : insérer une capture d'écran VSCode montrant la tâche cron qui identifie les investissements INITIATED trop anciens (via `updatedAt/createdAt`) et les marque en échec.

Capture d'écran

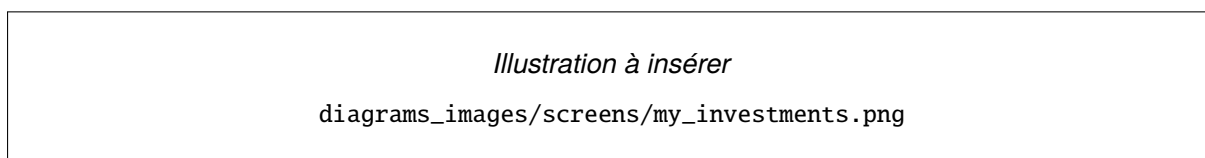


FIGURE 4.7 – Page « Mes investissements » côté investisseur (statuts INITIATED, SUCCESS, FAILED)

Composants techniques associés

Service interne `cronService`, file BullMQ pour la production des notifications associées, journalisation structurée des transitions de statut.

4.3.5 Moteur de recommandation

Description fonctionnelle

Le moteur est implémenté dans `recommendationService.js` (`getRecommendationsForUser`). Il est **entièrement déterministe** dans la version livrée : sélection des projets ACTIVE non archivés, calcul d'un score par projet à partir de quatre composantes (pondérations codées en dur) — adéquation aux catégories préférées du profil, adéquation au `riskLevel` issu de `project.aiAnalysis`, popularité (progression vers l'objectif) et récence de création — puis tri décroissant et découpe selon la limite demandée (défaut 8, plafonnée à 20). Aucun appel LLM n'est effectué dans ce service : l'IA intervient en amont lors de l'analyse de projet, dont le résultat est réutilisé ici.

Bibliothèques utilisées

— `mongoose` — requêtes `find` sur `projects` et `users` (`lean()`).

Capture VSCode à insérer : calcul du score de recommandation

À faire : insérer une capture d'écran VSCode montrant la fonction de scoring déterministe (catégories, riskLevel, progression, récence) et le tri/découpe.

Capture d'écran

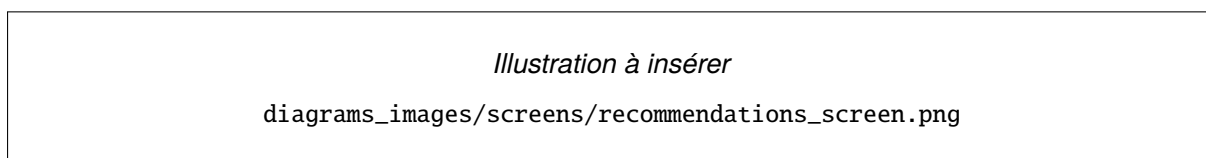


FIGURE 4.8 – Page d'accueil : projets recommandés en fonction du profil de l'investisseur

Composants techniques associés

Cache mémoire de courte durée pour les recommandations par utilisateur, *fallback* déterministe en cas d'indisponibilité de l'IA, journalisation des explications retournées par le LLM.

4.3.6 Service de notifications

Description fonctionnelle

Le service de notifications enregistre systématiquement les notifications **in-app** dans MongoDB (lecture depuis le frontend). Pour les événements critiques (sécurité, paiements, *payouts*, décisions sur projets), un e-mail est également envoyé, via une file d'attente (BullMQ/Redis) et/ou un workflow n8n selon le type d'événement. Les mécanismes de reprise (réessais, journalisation des échecs) permettent de conserver une traçabilité et d'éviter la perte d'informations.

Bibliothèques utilisées

- mongoose — collection notifications (in-app), index sur userId et readAt.
- bullmq et ioredis — files email-queue et notification-queue.
- nodemailer — envoi SMTP via le *worker* BullMQ.
- n8n — orchestration des workflows d'analyse de risque; le chatbot et la modération passent principalement par l'API Express.

Capture VSCode à insérer : notification critique (in-app + e-mail)

À faire : insérer une capture d'écran VSCode montrant l'émission d'une notification in-app et l'ajout d'un job BullMQ pour l'e-mail (tentatives + backoff).

Capture d'écran

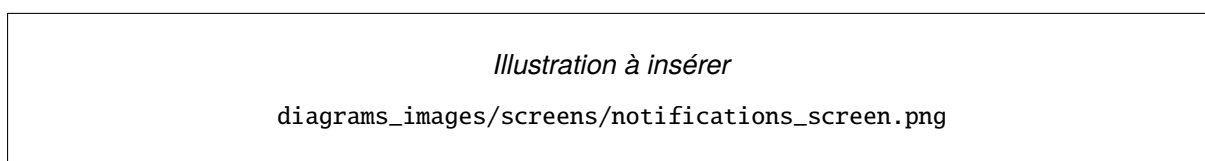


FIGURE 4.9 – Panneau in-app de notifications (icône cloche, statuts lus / non lus)

Composants techniques associés

DLQ `failedWorkflowEvents` pour les événements non récupérables, console d'administration permettant la reprise manuelle, politique de filtrage « critique ON / bruit OFF » côté e-mail.

4.3.7 Stratégie de test

La validation a été effectuée au moyen de **scripts E2E Node.js** qui simulent des parcours métier complets via l'API, en l'absence de *frameworks* de tests unitaires (le projet n'utilise ni Jest, ni Mocha, ni Cypress) :

- `e2e-lifecycle-payout-api.js` : inscription avec vérification OTP, création projet (DRAFT), soumission à l'analyse IA, validation et publication par l'administrateur, contribution + *webhook* signé, création du *payout*, saisie des coordonnées bancaires chiffrées, approbation administrateur.
- `e2e-lifecycle-scenarios.js` et `e2e-edgecases-api.js` : scénarios limites (annulation d'investissement, relance de paiement, sur-financement et remboursement, transitions de statut invalides, erreurs de validation).
- `e2e-interactions.js` : interactions sociales (commentaires, signalements, chatbot).

En complément, des **tests manuels guidés** ont été réalisés sur l'interface web (parcours porteur de projet, investisseur, administrateur) afin de vérifier l'ergonomie, la cohérence des messages et la fidélité de l'affichage des statuts. Les scripts de *smoke test* (`scripts/smoke.ps1`) permettent de valider rapidement la disponibilité des principaux endpoints après chaque déploiement.

4.3.8 Déploiement

La plateforme est conçue pour être déployée sur un environnement cloud managé (ou sur un serveur institutionnel) avec séparation des composants :

- **Frontend** : application React servie via un hébergement web (HTTPS).
- **Backend** : service Node.js/Express exposant l'API REST.
- **Base de données** : MongoDB (replica set requis pour les transactions multi-documents).
- **Broker** : Redis pour les files BullMQ.
- **Orchestration** : instance n8n (auto-hébergée) pour les workflows.

Les exigences de sécurité (HTTPS, secrets gérés hors code, signatures *webhook*, chiffrement AES-256-GCM des RIB) sont identiques en local et en production ; seules les valeurs des secrets et l'origine CORS varient par environnement.

4.4 Conclusion

L'implémentation a permis de valider les principaux processus métier de la plateforme : cycle de vie complet des projets (DRAFT → AWAITING_AI → UNDER_REVIEW → APPROVED → ACTIVE → FUNDED / CLOSED / SUSPENDED, avec branche REJECTED), contribution avec paiement simulé idempotent, analyse de risque et recommandations, gestion résiliente des échecs (DLQ), notifications critiques et administration. La cohérence de bout en bout a été vérifiée par des scripts E2E Node.js et complétée par des tests manuels guidés sur l'interface. La conclusion générale qui suit met en perspective les apports, les limites et les pistes d'évolution.

Conclusion générale

Travaux réalisés et solutions apportées

Ce projet a abouti à la réalisation d'une plateforme web de financement participatif couvrant les processus métier essentiels : création et gestion des campagnes, contribution des investisseurs, gouvernance par validation administrative, gestion des retraits, ainsi que des mécanismes de transparence (rapport d'analyse de risque, notifications, modération). Les principales réalisations sont :

- Un **parcours complet** porteur de projet : brouillon, soumission à l'analyse, revue/validation, publication, suivi de la collecte, puis préparation du retrait.
- Un **parcours complet** investisseur : exploration, contribution, suivi des investissements et gestion des incidents (paiement échoué, annulation, remboursement).
- Des **aides à la décision** : analyse de risque produisant un rapport structuré et recommandations adaptées au profil.
- Une **gouvernance et conformité** renforcées : rôles, modération, signalements, journalisation des actions et notifications des événements critiques.

Sur le plan technique, plusieurs solutions structurantes ont été conçues pour traiter les difficultés rencontrées : un *webhook* de paiement **signé en HMAC et idempotent** grâce à un index unique (`provider`, `providerPaymentId`) ; une **mise à jour conditionnelle atomique** de la collection `projects` (`currentFunding` + montant $\leq$ `fundingGoal`) qui préserve l'intégrité du financement sous charge concurrente ; un **repli déterministe** pour le moteur de recommandation et un cron `retryStuckAiAnalyses` pour les projets bloqués en `AWAITING_AI` en cas d'indisponibilité du LLM ; quatre files *dead-letter* (`FailedWorkflowEvents`, `FailedRefund`, `FailedPayoutEvents`, `FailedCancellationEvents`) qui rendent visible toute erreur définitive dans les chaînes asynchrones n8n / BullMQ et permettent une reprise manuelle ; enfin, le **chiffrement AES-256-GCM** des RIB avec un IV unique par enregistrement assure la confidentialité des coordonnées bancaires.

Au-delà de l'implémentation, le projet a mis en œuvre une démarche complète d'**ingénierie des systèmes d'information** : capture des besoins, modélisation UML, conception des données et validation par scénarios métier.

Apports personnels et compétences acquises

Ce projet de fin d'études a constitué une expérience formatrice à plusieurs niveaux :

- **Développement full-stack** : conception d'une SPA React et d'API REST Express, avec séparation claire entre logique métier et accès aux données.
- **Sécurité applicative** : authentification par cookies HttpOnly, contrôle d'accès par rôles, *rate limiting*, signatures *webhook* et chiffrement applicatif.
- **Données et intégrité** : modélisation MongoDB, indexation, transactions multi-documents et gestion des cas critiques (sur-financement, remboursements, retraits).
- **IA appliquée au métier** : intégration d'un LLM dans un workflow n8n pour produire un rapport d'analyse de risque et alimenter un assistant de consultation projet, avec gestion des indisponibilités.
- **Analyse et modélisation** : formalisation des besoins, cas d'utilisation, diagrammes de séquence et de classes pour structurer le système d'information.
- **Validation** : conception de scénarios E2E bout en bout et contrôles de cohérence systématiques sur les statuts, transitions et messages utilisateurs.

Cette période de stage m'a permis de mettre en pratique les connaissances théoriques acquises durant la licence, tout en développant une autonomie dans la gestion d'un projet complexe. La collaboration régulière avec mon encadreur a été essentielle pour valider les choix techniques et débloquer certaines situations problématiques. L'aspect le plus enrichissant a été l'intégration de composants d'intelligence artificielle dans un système métier concret, me permettant de comprendre les enjeux pratiques de l'IA (performance, précision, gestion des erreurs) au-delà des aspects théoriques.

Critiques et limites

Malgré les résultats obtenus, certaines limites subsistent :

- Le moteur de recommandation est principalement fondé sur les préférences et des règles de compatibilité (catégories/risque) ; l'absence de données historiques limite l'évaluation

empirique.

- L'analyse de risque reste dépendante de la qualité de la description du projet ; une revue humaine demeure nécessaire, surtout pour les projets sensibles.
- La plateforme n'inclut pas de version mobile native, bien que le site soit *responsive*.
- Les validations ont été réalisées par scénarios automatisés côté API et tests manuels ; une automatisation IHM (tests UI) renforcerait encore la qualité.
- L'intégration d'un prestataire de paiement réel implique des exigences supplémentaires (KYC, *sandbox*, certificats, supervision) qui dépassent le périmètre PFE.

Perspectives

Plusieurs améliorations peuvent être envisagées pour enrichir le système :

- **Industrialiser le paiement** : intégrer un prestataire réel (*sandbox*), renforcer l'observabilité des *webhooks* et formaliser les contrôles de conformité.
- **Améliorer la décision** : affiner l'analyse de risque (meilleure explication, calibration) et enrichir les recommandations par apprentissage dès qu'un historique existe.
- **Pilotage SI** : ajouter des tableaux de bord (KPIs) orientés métier (taux de succès, délais de validation, volumes de contribution, incidents).
- **Qualité** : automatiser des tests UI et renforcer les tests de non-régression sur le cycle de vie projet/paiement/*payout*.

Ce travail illustre par ailleurs l'intérêt d'intégrer l'IA dans un système d'information métier afin d'améliorer la transparence (rapport lisible) et l'aide à la décision (recommandations), tout en conservant une gouvernance humaine (validation et modération) ; l'approche proposée peut être transposée à d'autres SI où la décision doit être **expliquée** et **traçable**.

Webographie

- Banque Centrale de Tunisie (BCT) — Cadre légal du financement participatif (Tunisie). <https://www.bct.gov.tn/>
- Object Management Group — UML 2.5.1 Specification. <https://www.omg.org/spec/UML/>
- OWASP — Top 10 Web Application Security Risks. <https://owasp.org/Top10/>
- React — Documentation officielle. <https://react.dev/>
- Node.js — Documentation officielle. <https://nodejs.org/>
- MongoDB — Documentation officielle. <https://www.mongodb.com/docs/>
- n8n — Documentation officielle. <https://n8n.io/>
- Scrum Guide — Guide officiel Scrum. <https://scrumguides.org/>

Annexe

Format type d'une description textuelle de cas d'utilisation

Le tableau 4.2 reprend le modèle utilisé au chapitre 2 pour les fiches détaillées (section 2.5.2) et, le cas échéant, pour les autres cas listés au tableau 2.1.

Rubrique	Contenu attendu
Cas d'utilisation	Nom métier (verbe + objet, ex. <i>Soutenir un projet</i>).
Acteur(s)	Rôles métier déclencheurs et participants.
Précondition	État du système requis avant l'exécution.
Postcondition	État du système garanti après une exécution réussie.
Scénario nominal	Séquence numérotée des interactions principales.
Scénarios alternatifs	Variantes ou branches optionnelles du nominal.
Exceptions	Cas d'erreur et leur traitement.

TABLE 4.2 – Modèle de description textuelle d'un cas d'utilisation

Bibliographie

- [1] Banque Centrale de Tunisie. (2020). *Loi n° 2020-37 du 2 juillet 2020 relative au financement participatif*. Journal Officiel de la République Tunisienne, n° 76.
- [2] Object Management Group. (2017). *Unified Modeling Language (UML) Specification, Version 2.5.1*. Consulté le 3 mai 2026. <https://www.omg.org/spec/UML/>
- [3] Belleflamme, P., Lambert, T., & Schwienbacher, A. (2014). Crowdfunding : Tapping the right crowd. *Journal of Business Venturing*, 29(5), 585-609.
- [4] Mollick, E. (2014). The dynamics of crowdfunding : An exploratory study. *Journal of Business Venturing*, 29(1), 1–16.
- [5] Ricci, F., Rokach, L., & Shapira, B. (2011). *Introduction to Recommender Systems Handbook*. Springer.
- [6] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3e éd.). Prentice Hall.
- [7] Zhang, J., & Liu, P. (2012). Rational herding in microloan markets. *Management Science*, 58(5), 892-912.
- [8] Greenberg, M. D., Pardo, B., Hariharan, K., & Gerber, E. (2013). Crowdfunding support tools : Predicting success & failure. In *CHI'13 Extended Abstracts on Human Factors in Computing Systems* (pp. 1815-1820).
- [9] Lukkarinen, A., Teich, J. E., Wallenius, H., & Wallenius, J. (2016). Success drivers of online equity crowdfunding campaigns. *Decision Support Systems*, 87, 26-38.
- [10] Cortellazzo, L., Bonesso, S., Gerli, F., & Batista-Foguet, J. M. (2020). Prototypicality and authenticity in crowdfunding : An fsQCA configurational approach. *International Journal of Research in Marketing*, 37(4), 721-738.
- [11] Meta Platforms, Inc. (2024). *React Documentation*. Consulté le 3 mai 2026. <https://react.dev/>
- [12] OpenJS Foundation. (2024). *Node.js Documentation*. Consulté le 3 mai 2026. <https://nodejs.org/>

- [13] MongoDB, Inc. (2024). *MongoDB Manual*. Consulté le 3 mai 2026. <https://www.mongodb.com/docs/>
- [14] n8n GmbH. (2024). *n8n Workflow Automation Documentation*. Consulté le 3 mai 2026. <https://n8n.io/>
- [15] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Consulté le 3 mai 2026. <https://scrumguides.org/>
- [16] Beck, K., Beedle, M., van Bennekum, A., et al. (2001). *Manifesto for Agile Software Development*. Consulté le 3 mai 2026. <https://agilemanifesto.org/>
- [17] OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*. Consulté le 3 mai 2026. <https://owasp.org/Top10/>
- [18] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) - RFC 7519*. Internet Engineering Task Force (IETF).